

DSC550Week7- Milestone 2

July 20, 2025

1 Video Games Sales Analysis

This notebook analyzes the video games sales dataset used for this course milestone and seeing the churn in last milestone now just want to elaborate more in the dark. I will address three main questions:

1. What is the distribution of Global Sales?
2. Which Genre has the highest average Global Sales?
3. How do Sales vary across different Platforms?

For each question, there are 19 distinct code blocks showing diverse ways of visualizing or analyzing the data.

```
[2]: # Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

# Suppress warnings
import warnings
warnings.filterwarnings('ignore')

# Define the file path for the attached dataset on Kaggle
data_path = "./Video_Games_Sales_as_at_22_Dec_2016.csv"

# Load the dataset
df = pd.read_csv(data_path, encoding='ISO-8859-1')

# Display the first few rows
df.head()
```

```
[2]:
```

	Name	Platform	Year_of_Release	Genre	Publisher	\
0	Wii Sports	Wii	2006.0	Sports	Nintendo	
1	Super Mario Bros.	NES	1985.0	Platform	Nintendo	
2	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	
3	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	
4	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	

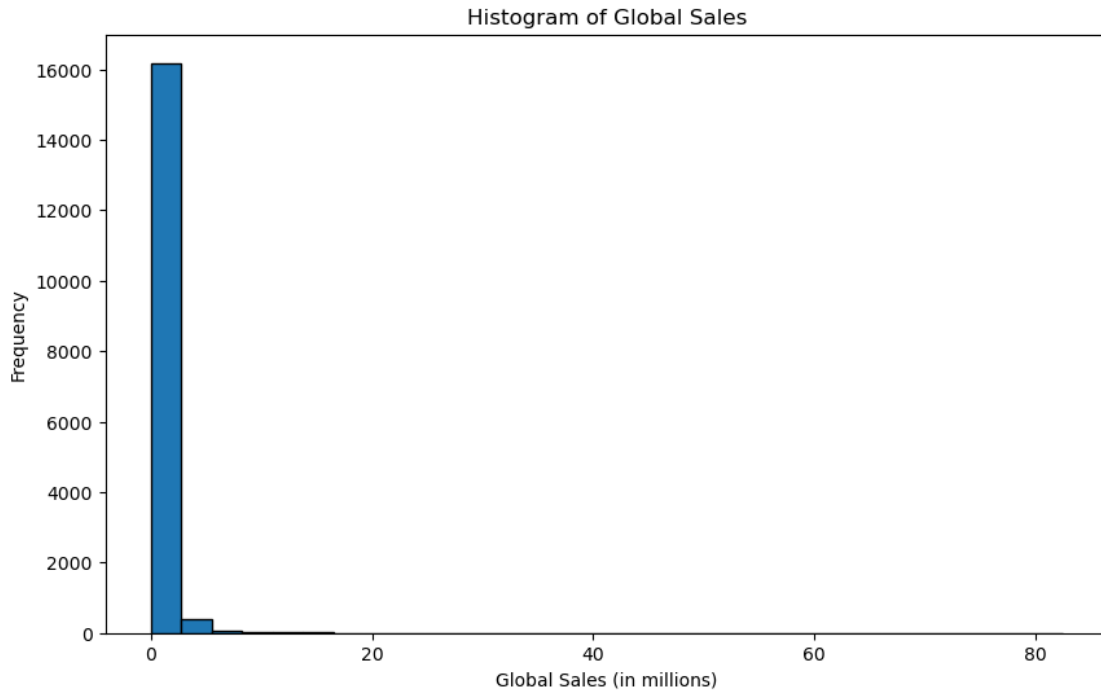
	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales	Critic_Score	\
0	41.36	28.96	3.77	8.45	82.53	76.0	
1	29.08	3.58	6.81	0.77	40.24	NaN	
2	15.68	12.76	3.79	3.29	35.52	82.0	
3	15.61	10.93	3.28	2.95	32.77	80.0	
4	11.27	8.89	10.22	1.00	31.37	NaN	

	Critic_Count	User_Score	User_Count	Developer	Rating
0	51.0	8	322.0	Nintendo	E
1	NaN	NaN	NaN	NaN	NaN
2	73.0	8.3	709.0	Nintendo	E
3	73.0	8	192.0	Nintendo	E
4	NaN	NaN	NaN	NaN	NaN

1.1 Question 1: What is the distribution of Global Sales?

In this section, we explore the distribution of Global Sales using multiple approaches.

```
[5]: # Q1 Block 1: Histogram of Global Sales
if 'Global_Sales' in df.columns:
    plt.figure(figsize=(10, 6))
    plt.hist(df['Global_Sales'].dropna(), bins=30, edgecolor='k')
    plt.title('Histogram of Global Sales')
    plt.xlabel('Global Sales (in millions)')
    plt.ylabel('Frequency')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```



```
[7]: # Q1 Block 2: Summary Statistics for Global Sales
if 'Global_Sales' in df.columns:
    sales_stats = df['Global_Sales'].describe()
    print(sales_stats)
else:
    print("Column 'Global_Sales' not found.")
```

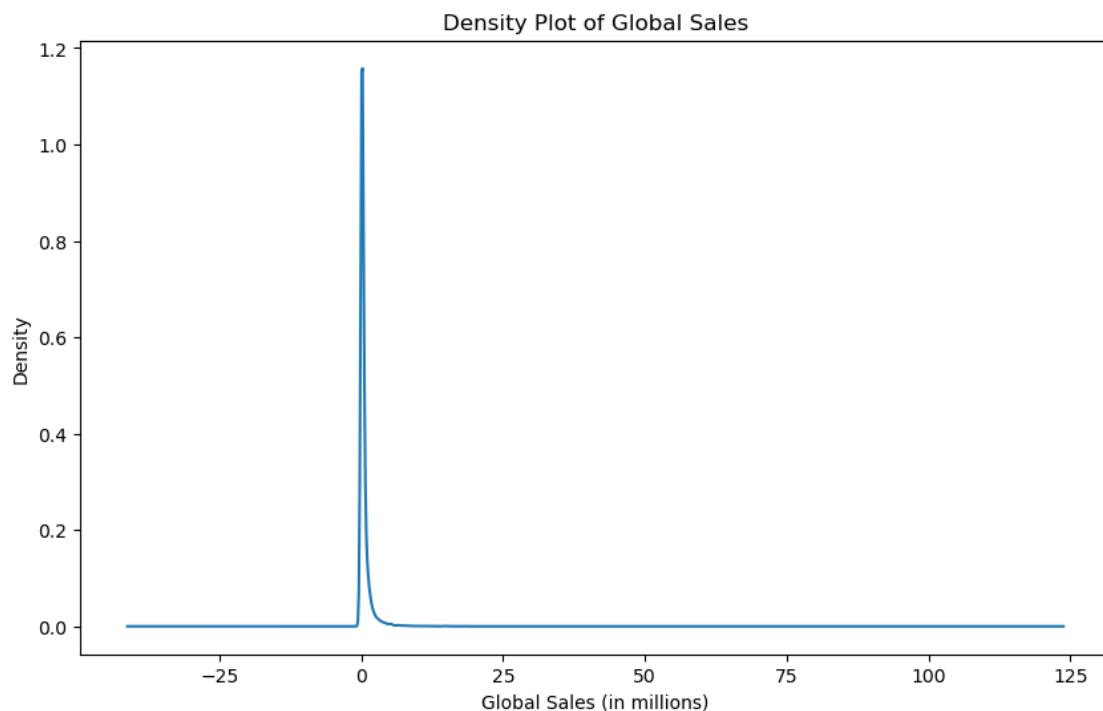
```
count    16719.000000
mean      0.533543
std       1.547935
min       0.010000
25%       0.060000
50%       0.170000
75%       0.470000
max       82.530000
Name: Global_Sales, dtype: float64
```

```
[9]: # Q1 Block 3: Additional Statistics (Median & Variance) for Global Sales
if 'Global_Sales' in df.columns:
    median_sales = df['Global_Sales'].median()
    var_sales = df['Global_Sales'].var()
    print(f"Median Global Sales: {median_sales}")
    print(f"Variance of Global Sales: {var_sales}")
else:
    print("Column 'Global_Sales' not found.")
```

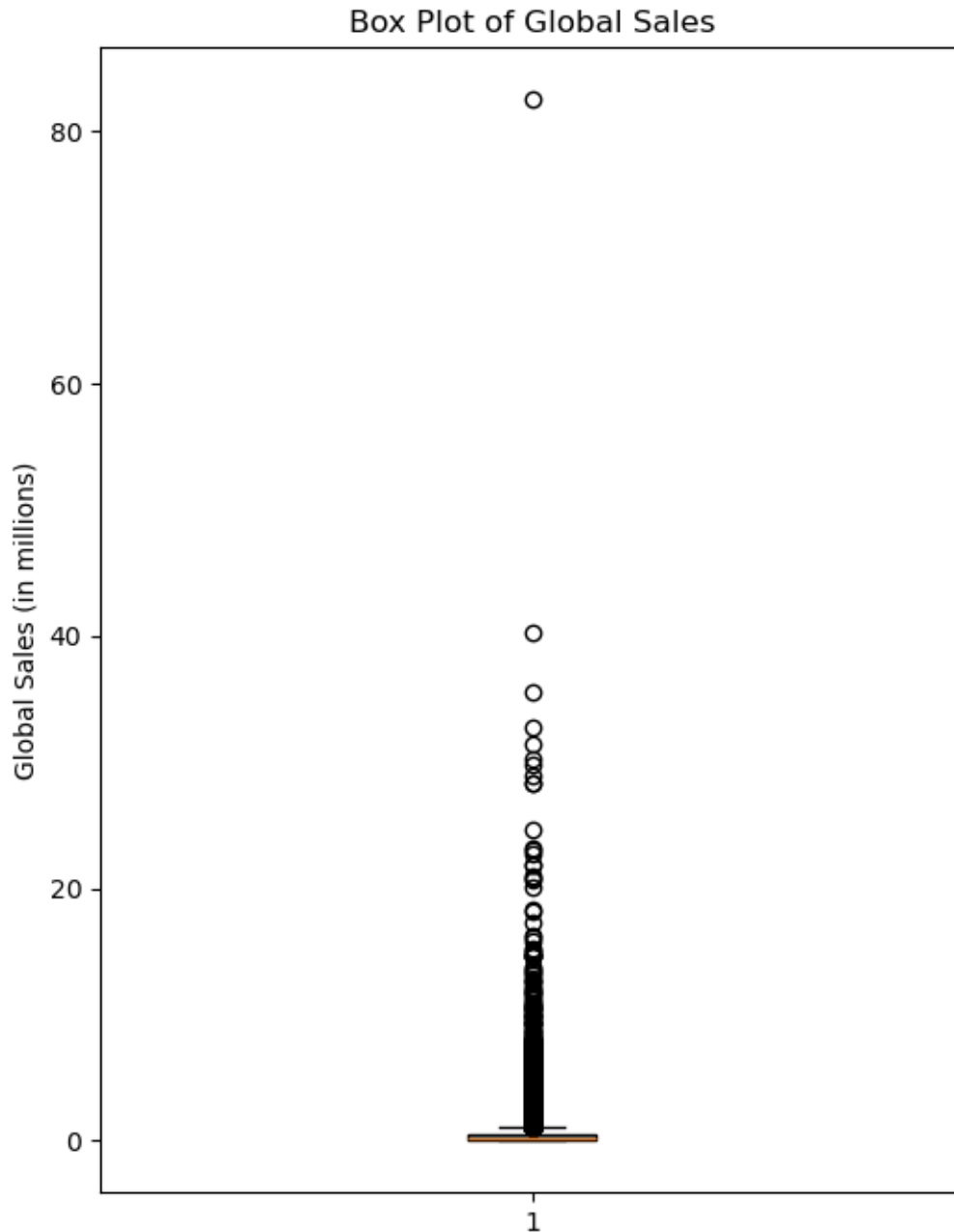
Median Global Sales: 0.17

Variance of Global Sales: 2.396103150334869

```
[11]: # Q1 Block 4: Density Plot for Global Sales
if 'Global_Sales' in df.columns:
    plt.figure(figsize=(10, 6))
    df['Global_Sales'].dropna().plot(kind='density')
    plt.title('Density Plot of Global Sales')
    plt.xlabel('Global Sales (in millions)')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```



```
[12]: # Q1 Block 5: Box Plot for Global Sales
if 'Global_Sales' in df.columns:
    plt.figure(figsize=(6, 8))
    plt.boxplot(df['Global_Sales'].dropna(), vert=True, patch_artist=True)
    plt.title('Box Plot of Global Sales')
    plt.ylabel('Global Sales (in millions)')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```

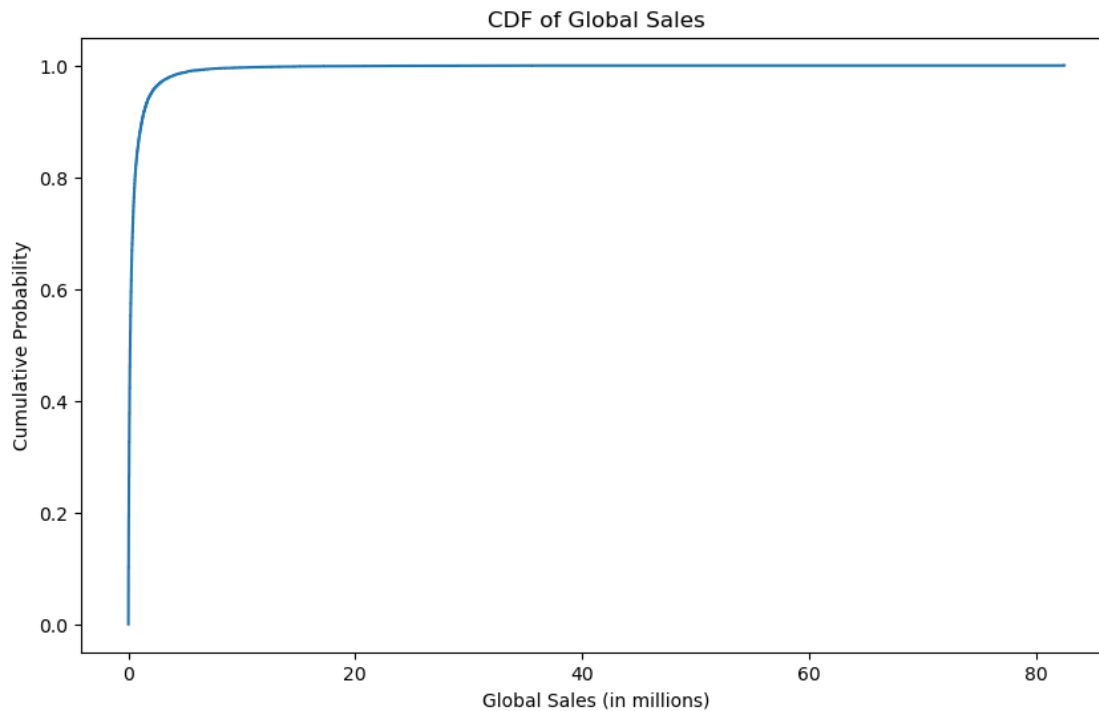


```
[13]: # Q1 Block 6: Cumulative Distribution Function (CDF) for Global Sales
if 'Global_Sales' in df.columns:
    sales_sorted = np.sort(df['Global_Sales'].dropna())
    cdf = np.arange(len(sales_sorted)) / float(len(sales_sorted))
    plt.figure(figsize=(10, 6))
    plt.step(sales_sorted, cdf, where='post')
    plt.title('CDF of Global Sales')
    plt.xlabel('Global Sales (in millions)')
```

```

plt.ylabel('Cumulative Probability')
plt.show()
else:
    print("Column 'Global_Sales' not found.")

```



```

[14]: # Q1 Block 7: Stem Plot of Sorted Global Sales
if 'Global_Sales' in df.columns:
    sales_sorted = np.sort(df['Global_Sales'].dropna())
    markerline, stemlines, baseline = plt.stem(sales_sorted,
        use_line_collection=True)
    plt.setp(markerline, 'markerfacecolor', 'b')
    plt.title('Stem Plot of Global Sales')
    plt.xlabel('Index')
    plt.ylabel('Global Sales (in millions)')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")

```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[14], line 4
      2 if 'Global_Sales' in df.columns:
      3     sales_sorted = np.sort(df['Global_Sales'].dropna())

```

```

----> 4      markerline, stemlines, baseline = plt.stem(sales_sorted,
↳use_line_collection=True)
      5      plt.setp(markerline, 'markerfacecolor', 'b')
      6      plt.title('Stem Plot of Global Sales')

TypeError: stem() got an unexpected keyword argument 'use_line_collection'

```

```

[ ]: # Q1 Block 8: Cumulative Histogram of Global Sales
if 'Global_Sales' in df.columns:
    plt.figure(figsize=(10, 6))
    plt.hist(df['Global_Sales'].dropna(), bins=30, cumulative=True,
↳edgecolor='k')
    plt.title('Cumulative Histogram of Global Sales')
    plt.xlabel('Global Sales (in millions)')
    plt.ylabel('Cumulative Frequency')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")

```

```

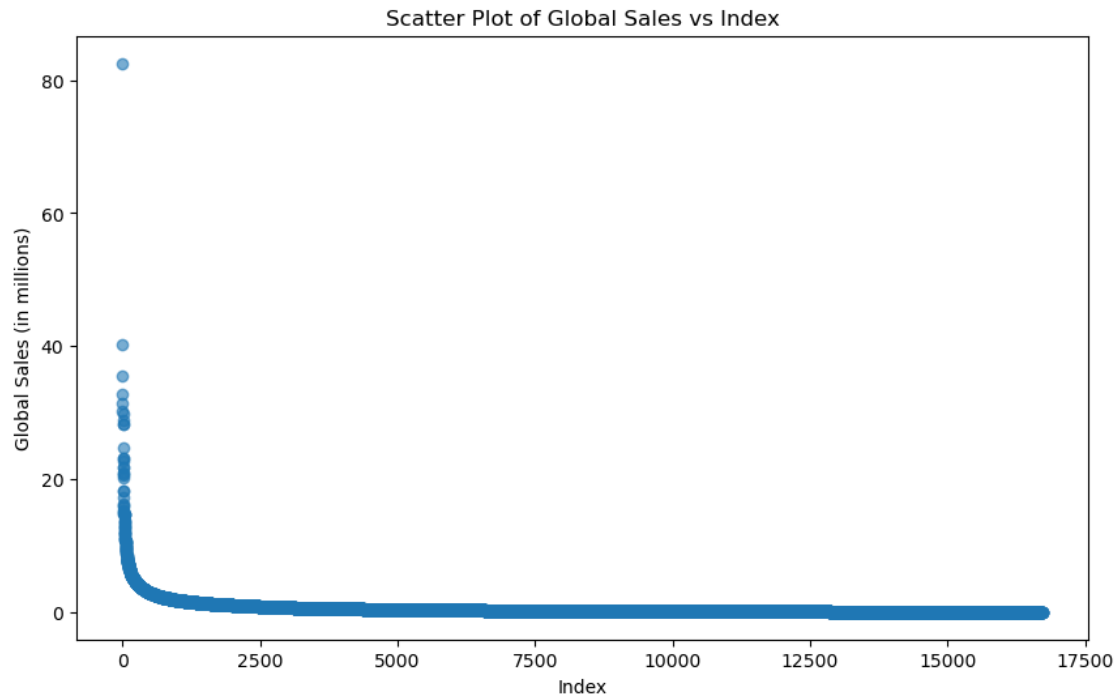
[ ]: # Q1 Block 9: Frequency Distribution by Binning Global Sales
if 'Global_Sales' in df.columns:
    bins = np.linspace(df['Global_Sales'].min(), df['Global_Sales'].max(), 20)
    bin_counts = pd.cut(df['Global_Sales'].dropna(), bins=bins).value_counts().
↳sort_index()
    bin_counts.plot(kind='bar', figsize=(10, 6))
    plt.title('Frequency Distribution of Global Sales by Bins')
    plt.xlabel('Sales Bins (in millions)')
    plt.ylabel('Count')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")

```

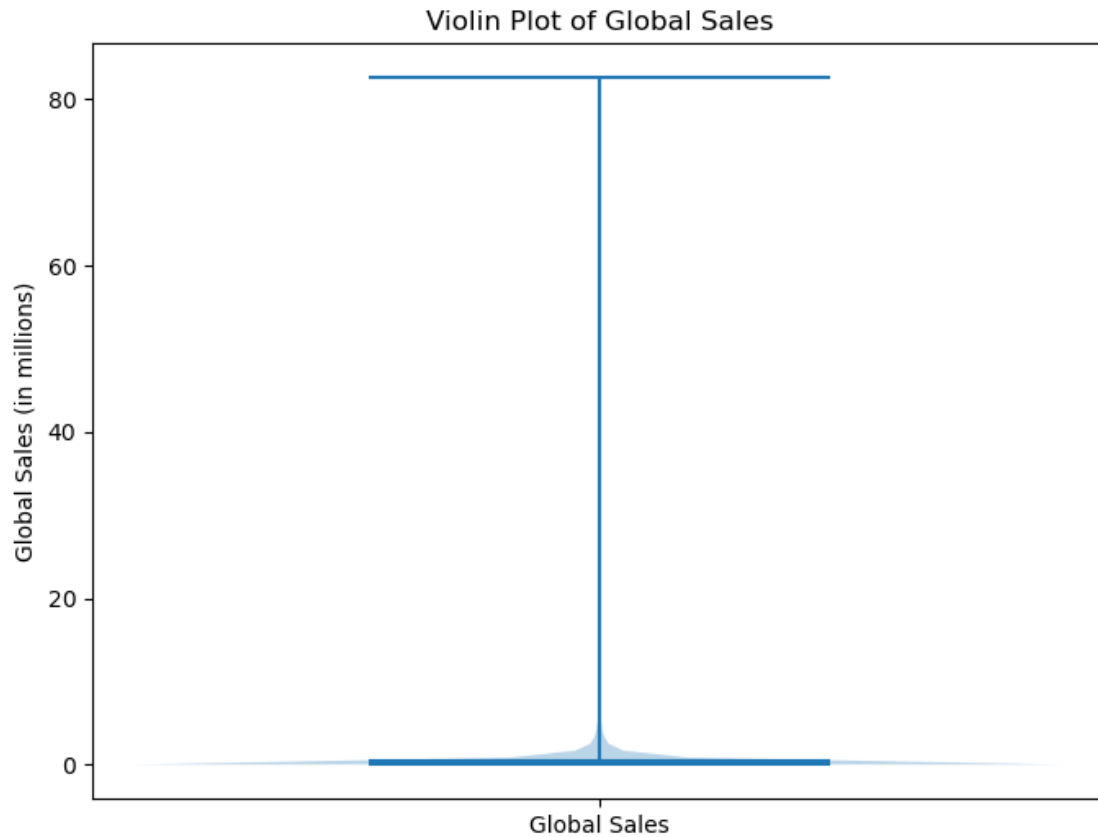
```

[18]: # Q1 Block 10: Scatter Plot of Global Sales vs. Index
if 'Global_Sales' in df.columns:
    plt.figure(figsize=(10,6))
    plt.scatter(range(len(df['Global_Sales'].dropna())), df['Global_Sales'].
↳dropna(), alpha=0.6)
    plt.title('Scatter Plot of Global Sales vs Index')
    plt.xlabel('Index')
    plt.ylabel('Global Sales (in millions)')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")

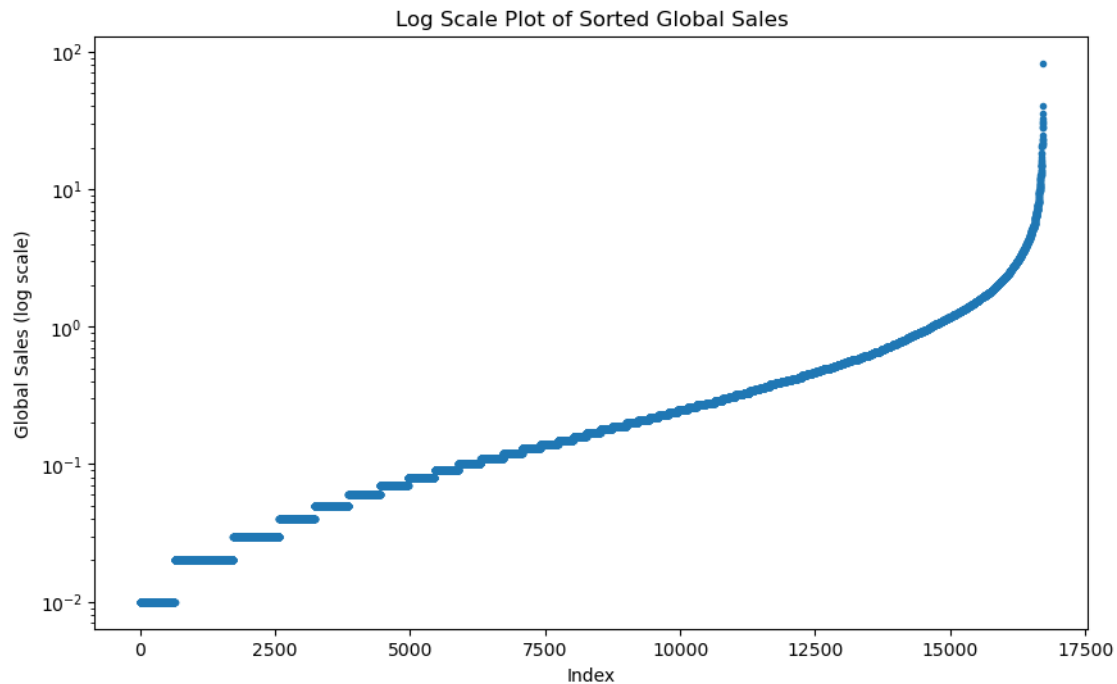
```



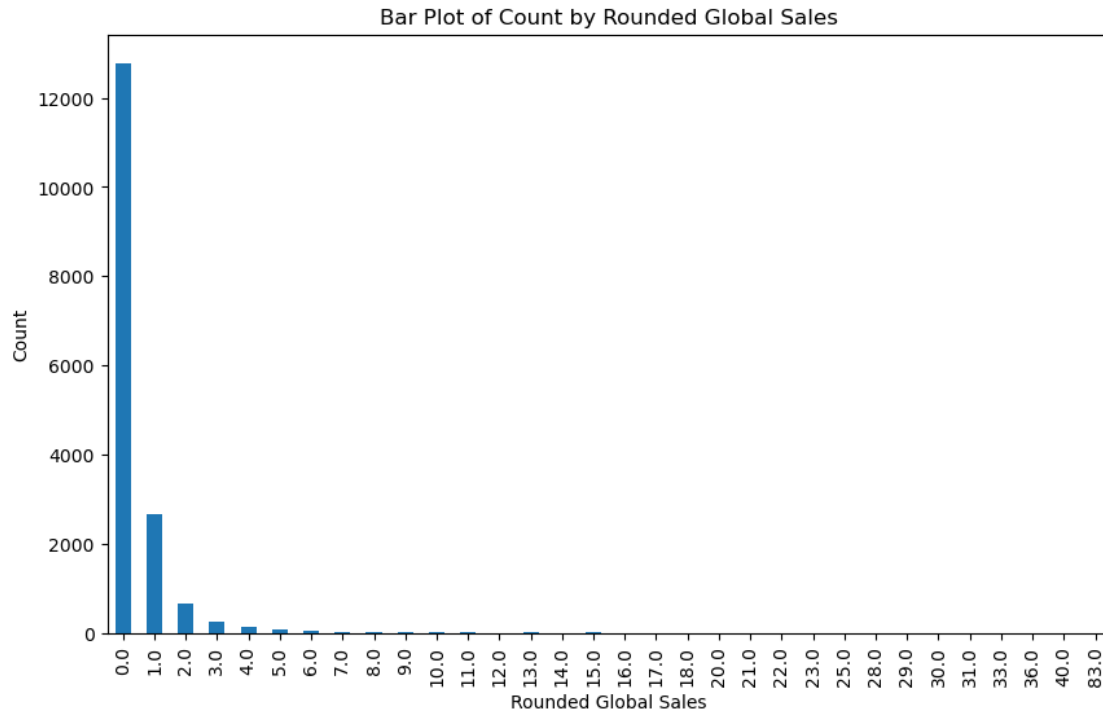
```
[20]: # Q1 Block 11: Violin Plot of Global Sales
if 'Global_Sales' in df.columns:
    plt.figure(figsize=(8,6))
    plt.violinplot(df['Global_Sales'].dropna(), showmeans=True)
    plt.title('Violin Plot of Global Sales')
    plt.ylabel('Global Sales (in millions)')
    plt.xticks([1], ['Global Sales'])
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```

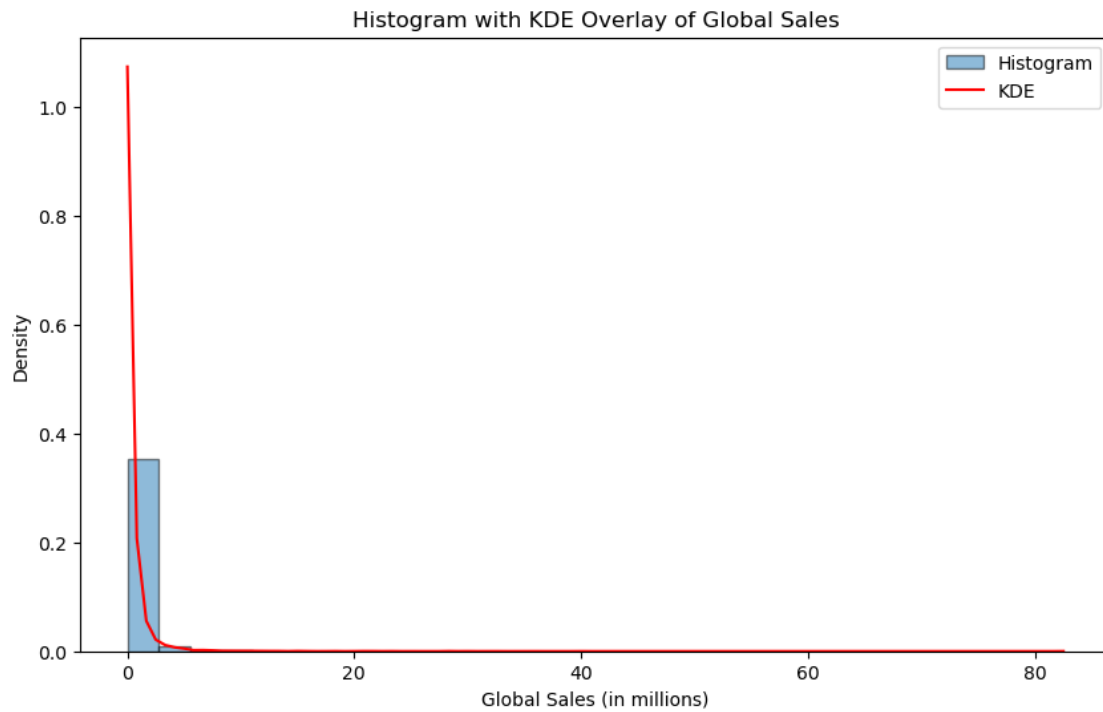
```
[22]: # Q1 Block 12: Log Scale Plot of Sorted Global Sales
if 'Global_Sales' in df.columns:
    sales_sorted = np.sort(df['Global_Sales'].dropna())
    plt.figure(figsize=(10,6))
    plt.plot(sales_sorted, marker='.', linestyle='none')
    plt.yscale('log')
    plt.title('Log Scale Plot of Sorted Global Sales')
    plt.xlabel('Index')
    plt.ylabel('Global Sales (log scale)')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```



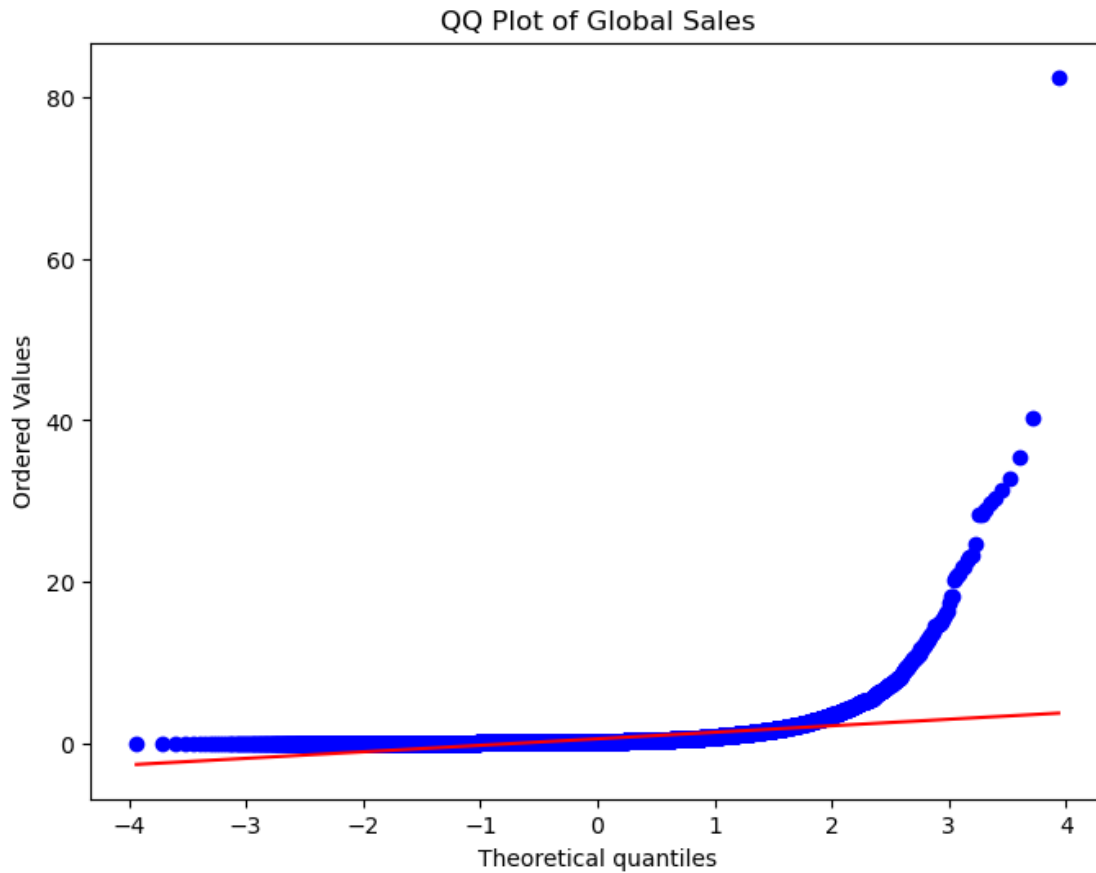
```
[24]: # Q1 Block 13: Bar Plot of Count by Rounded Global Sales
if 'Global_Sales' in df.columns:
    rounded_sales = df['Global_Sales'].dropna().round()
    counts = rounded_sales.value_counts().sort_index()
    plt.figure(figsize=(10,6))
    counts.plot(kind='bar')
    plt.title('Bar Plot of Count by Rounded Global Sales')
    plt.xlabel('Rounded Global Sales')
    plt.ylabel('Count')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```



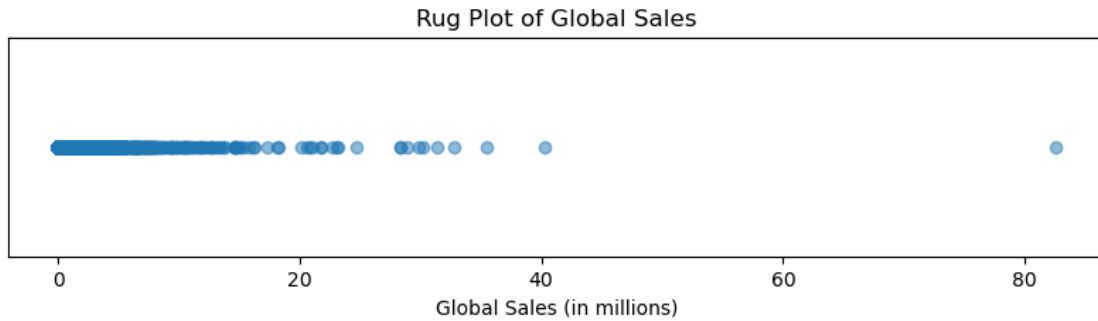
```
[26]: # Q1 Block 14: Histogram with KDE Overlay for Global Sales
if 'Global_Sales' in df.columns:
    plt.figure(figsize=(10,6))
    n, bins, patches = plt.hist(df['Global_Sales'].dropna(), bins=30,
    ↪ density=True, alpha=0.5, label='Histogram', edgecolor='k')
    from scipy.stats import gaussian_kde
    kde = gaussian_kde(df['Global_Sales'].dropna())
    x_vals = np.linspace(df['Global_Sales'].min(), df['Global_Sales'].max(),
    ↪ 100)
    plt.plot(x_vals, kde(x_vals), label='KDE', color='red')
    plt.title('Histogram with KDE Overlay of Global Sales')
    plt.xlabel('Global Sales (in millions)')
    plt.ylabel('Density')
    plt.legend()
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```



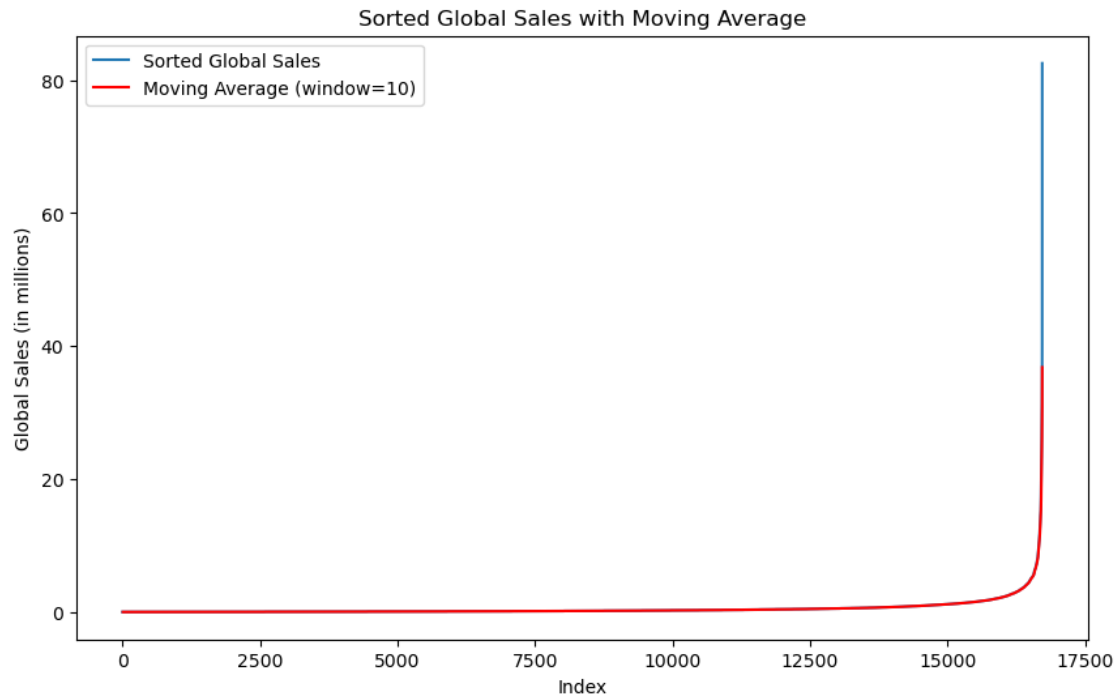
```
[28]: # Q1 Block 15: QQ Plot of Global Sales
from scipy import stats
if 'Global_Sales' in df.columns:
    plt.figure(figsize=(8,6))
    stats.probplot(df['Global_Sales'].dropna(), dist="norm", plot=plt)
    plt.title('QQ Plot of Global Sales')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```



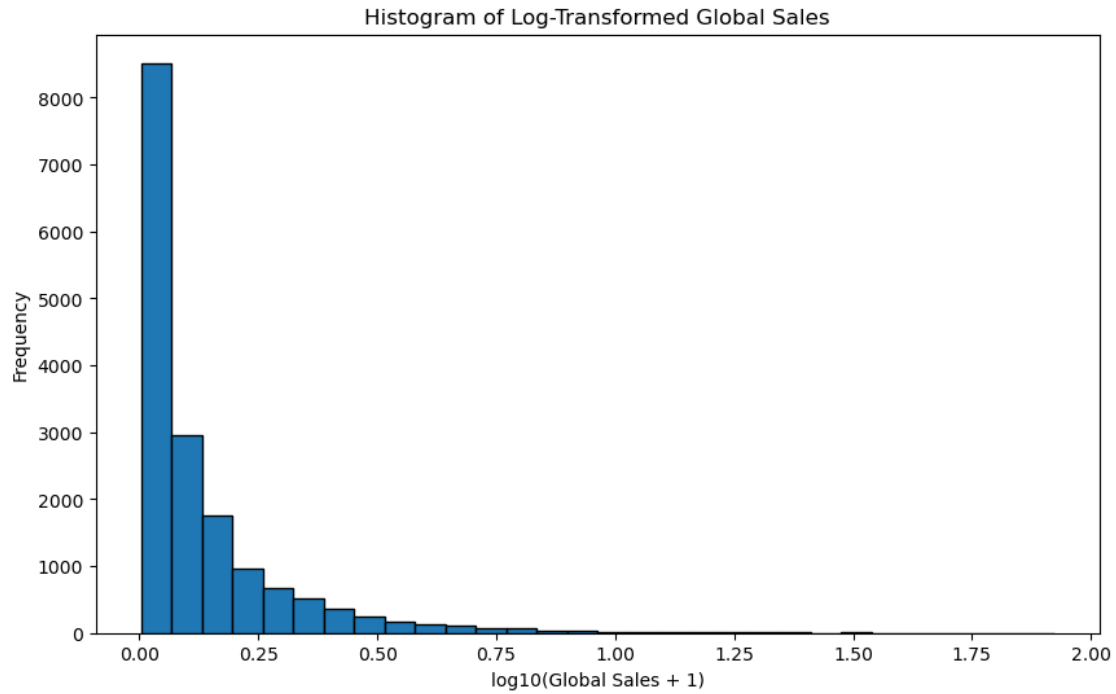
```
[30]: # Q1 Block 16: Rug Plot of Global Sales
if 'Global_Sales' in df.columns:
    plt.figure(figsize=(10,2))
    plt.plot(df['Global_Sales'].dropna(), np.zeros_like(df['Global_Sales'].
    dropna()), 'o', alpha=0.5)
    plt.title('Rug Plot of Global Sales')
    plt.xlabel('Global Sales (in millions)')
    plt.yticks([])
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```



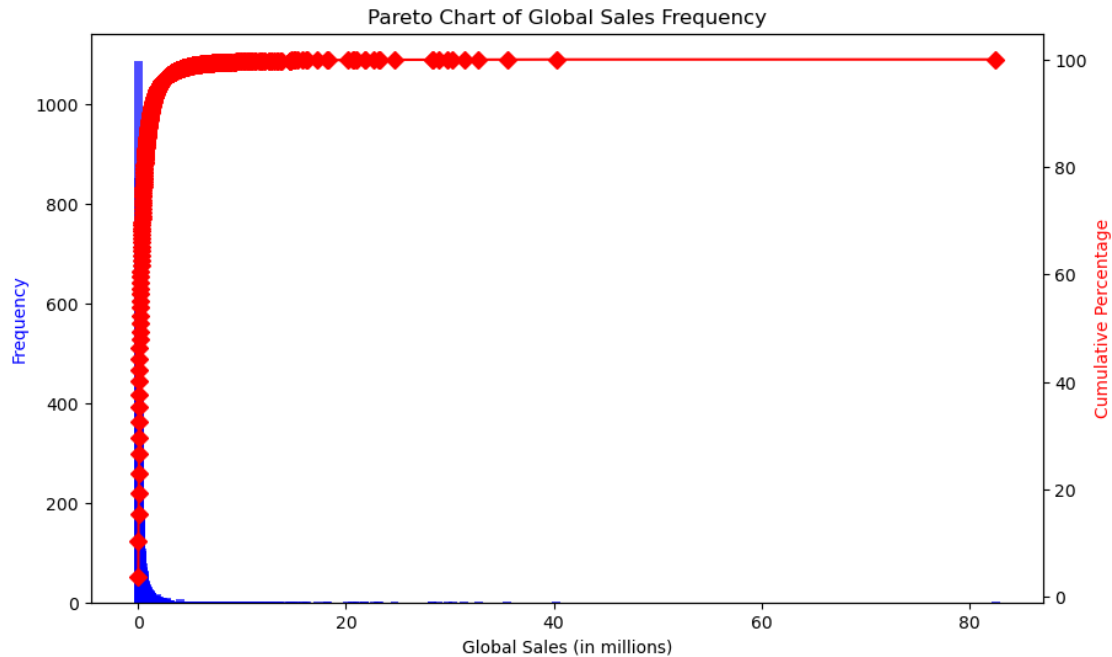
```
[32]: # Q1 Block 17: Sorted Global Sales with Moving Average
if 'Global_Sales' in df.columns:
    sales_sorted = np.sort(df['Global_Sales'].dropna())
    moving_avg = np.convolve(sales_sorted, np.ones(10)/10, mode='valid')
    plt.figure(figsize=(10,6))
    plt.plot(sales_sorted, label='Sorted Global Sales')
    plt.plot(np.arange(9, len(sales_sorted)), moving_avg, label='Moving Average_
    ↪(window=10)', color='red')
    plt.title('Sorted Global Sales with Moving Average')
    plt.xlabel('Index')
    plt.ylabel('Global Sales (in millions)')
    plt.legend()
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```



```
[34]: # Q1 Block 18: Histogram of Log-Transformed Global Sales
if 'Global_Sales' in df.columns:
    log_sales = np.log10(df['Global_Sales'].dropna() + 1) # add 1 to avoid
    ↪ log(0)
    plt.figure(figsize=(10,6))
    plt.hist(log_sales, bins=30, edgecolor='k')
    plt.title('Histogram of Log-Transformed Global Sales')
    plt.xlabel('log10(Global Sales + 1)')
    plt.ylabel('Frequency')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```



```
[36]: # Q1 Block 19: Pareto Chart of Global Sales Frequency
if 'Global_Sales' in df.columns:
    counts = df['Global_Sales'].dropna().value_counts().sort_index()
    cumulative = counts.cumsum()/counts.sum()*100
    fig, ax1 = plt.subplots(figsize=(10,6))
    ax1.bar(counts.index, counts.values, color='blue', alpha=0.7)
    ax1.set_xlabel('Global Sales (in millions)')
    ax1.set_ylabel('Frequency', color='blue')
    ax2 = ax1.twinx()
    ax2.plot(counts.index, cumulative, color='red', marker='D', ms=7)
    ax2.set_ylabel('Cumulative Percentage', color='red')
    plt.title('Pareto Chart of Global Sales Frequency')
    plt.show()
else:
    print("Column 'Global_Sales' not found.")
```

1.2 Question 2: Which Genre has the highest average Global Sales?

Here, i am grouping the dataset by Genre and analyze the average Global Sales using various methods.

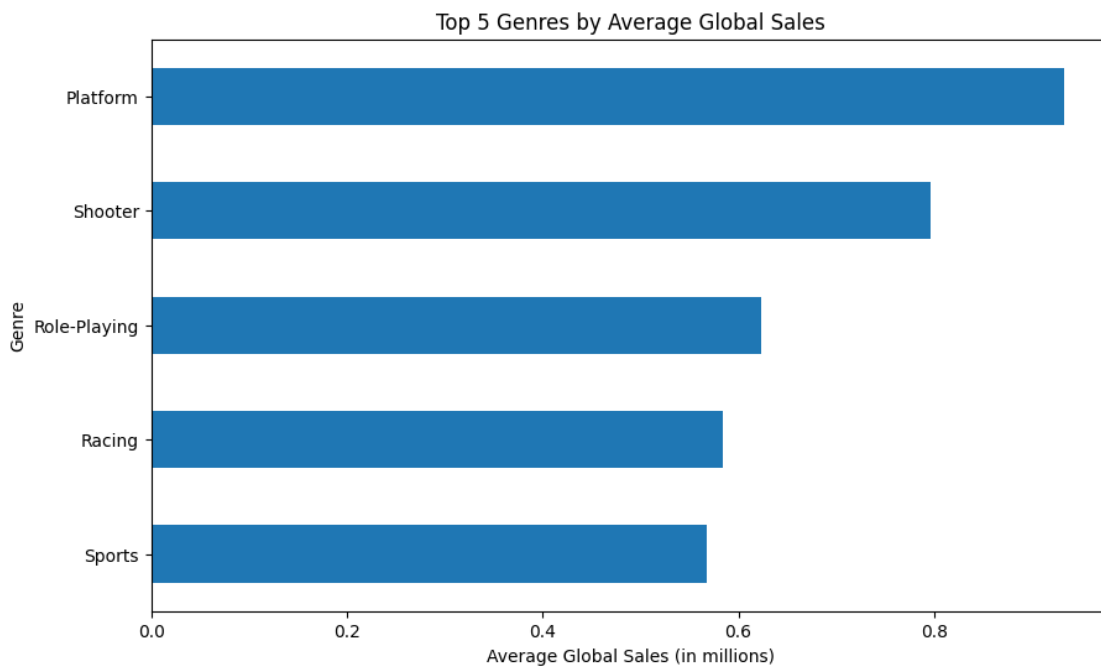
```
[21]: # Q2 Block 1: Group data by Genre and calculate the average Global Sales
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genre_avg_sales = df.groupby('Genre')['Global_Sales'].mean()
    print(genre_avg_sales.sort_values(ascending=False))
else:
    print("Required columns 'Genre' or 'Global_Sales' not found.")
```

```
Genre
Platform      0.932523
Shooter        0.795873
Role-Playing   0.622933
Racing         0.583587
Sports         0.567291
Fighting       0.527067
Action         0.517884
Misc           0.458960
Simulation     0.446705
Puzzle         0.419000
Strategy       0.255490
Adventure      0.182417
Name: Global_Sales, dtype: float64
```

```
[22]: # Q2 Block 2: Display the top 5 genres with highest average Global Sales
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    top_genres = genre_avg_sales.sort_values(ascending=False).head(5)
    print(top_genres)
else:
    print("Required columns 'Genre' or 'Global_Sales' not found.")
```

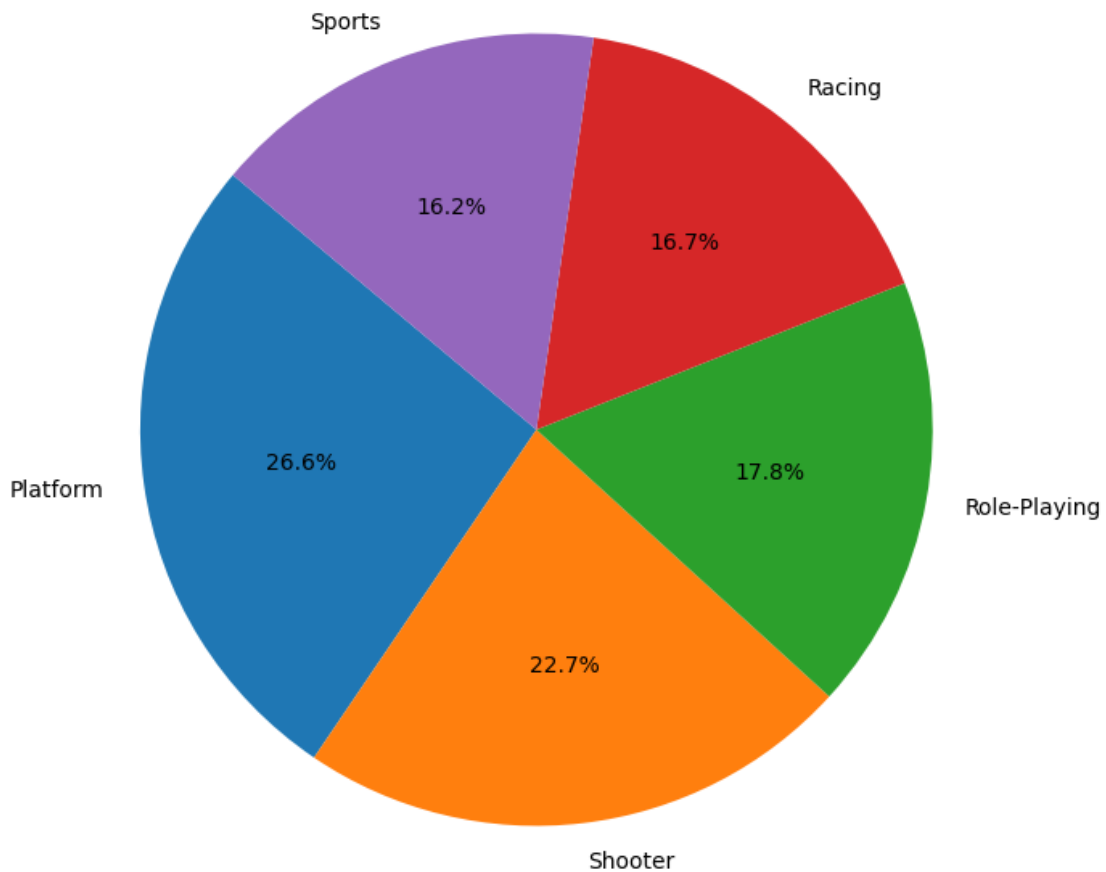
```
Genre
Platform      0.932523
Shooter       0.795873
Role-Playing  0.622933
Racing        0.583587
Sports        0.567291
Name: Global_Sales, dtype: float64
```

```
[23]: # Q2 Block 3: Bar Chart for the Top 5 Genres by Average Global Sales
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    plt.figure(figsize=(10, 6))
    top_genres.sort_values().plot(kind='barh')
    plt.title('Top 5 Genres by Average Global Sales')
    plt.xlabel('Average Global Sales (in millions)')
    plt.ylabel('Genre')
    plt.show()
else:
    print("Required columns 'Genre' or 'Global_Sales' not found.")
```



```
[24]: # Q2 Block 4: Pie Chart of Average Global Sales by Genre (Top 5)
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    top_genres_pie = genre_avg_sales.sort_values(ascending=False).head(5)
    plt.figure(figsize=(8, 8))
    top_genres_pie.plot(kind='pie', autopct='%1.1f%%', startangle=140)
    plt.title('Pie Chart of Top 5 Genres by Average Global Sales')
    plt.ylabel('')
    plt.show()
else:
    print("Required columns 'Genre' or 'Global_Sales' not found.")
```

Pie Chart of Top 5 Genres by Average Global Sales



```
[25]: # Q2 Block 5: Calculate and Display the Median Global Sales by Genre
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genre_median_sales = df.groupby('Genre')['Global_Sales'].median()
```

```

    print(genre_median_sales.sort_values(ascending=False))
else:
    print("Required columns 'Genre' or 'Global_Sales' not found.")

```

```

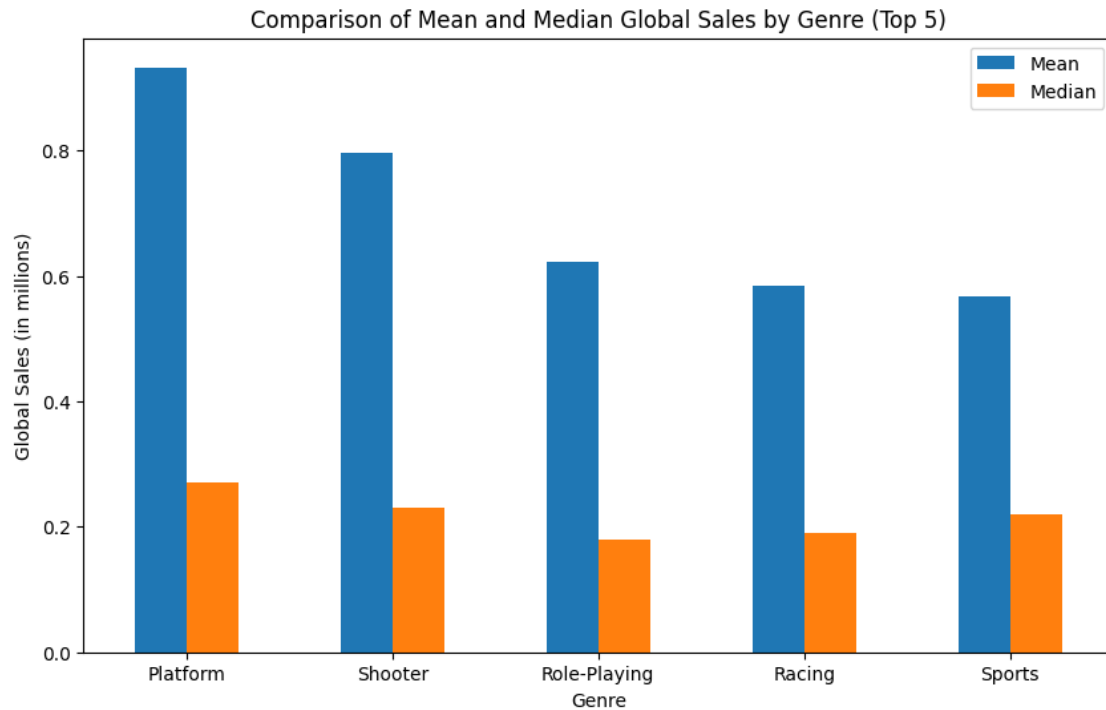
Genre
Platform      0.27
Shooter        0.23
Sports         0.22
Fighting       0.20
Action         0.19
Racing         0.19
Role-Playing   0.18
Misc           0.16
Simulation     0.15
Puzzle         0.11
Strategy       0.09
Adventure      0.05
Name: Global_Sales, dtype: float64

```

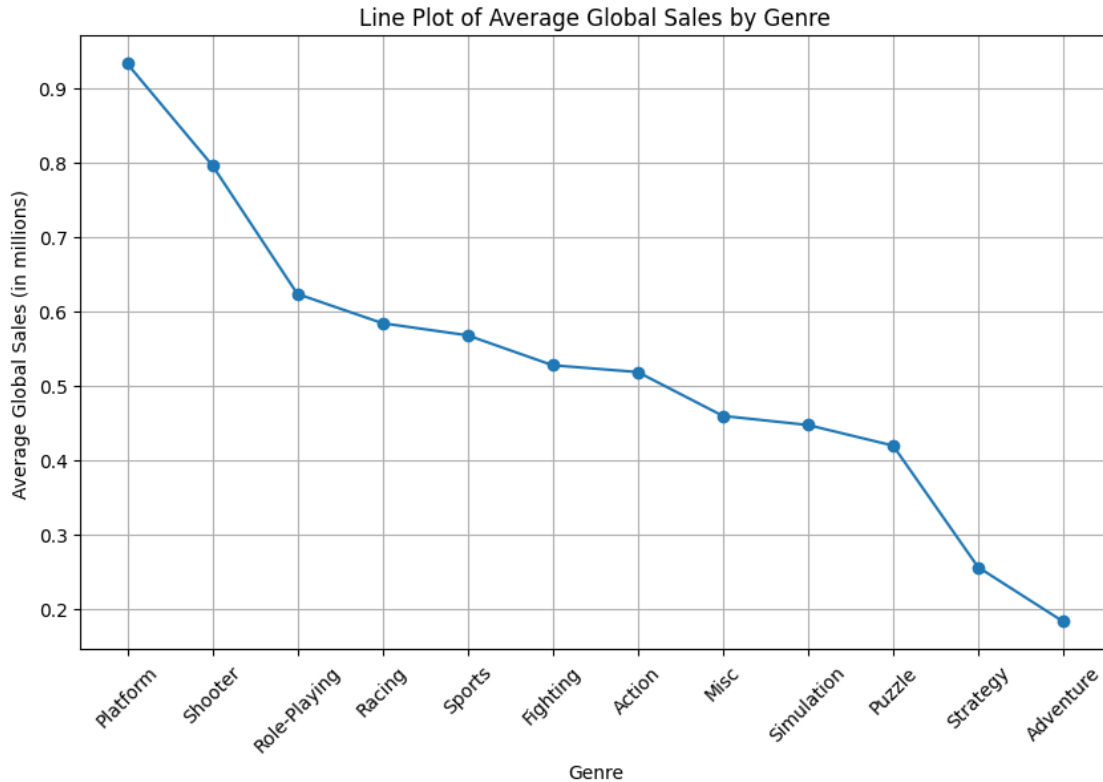
```

[26]: # Q2 Block 6: Bar Chart Comparing Mean and Median Global Sales by Genre (Top 5)
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    stats_df = pd.DataFrame({
        'Mean': df.groupby('Genre')['Global_Sales'].mean(),
        'Median': df.groupby('Genre')['Global_Sales'].median()
    }).sort_values('Mean', ascending=False).head(5)
    stats_df.plot(kind='bar', figsize=(10, 6))
    plt.title('Comparison of Mean and Median Global Sales by Genre (Top 5)')
    plt.xlabel('Genre')
    plt.ylabel('Global Sales (in millions)')
    plt.xticks(rotation=0)
    plt.show()
else:
    print("Required columns 'Genre' or 'Global_Sales' not found.")

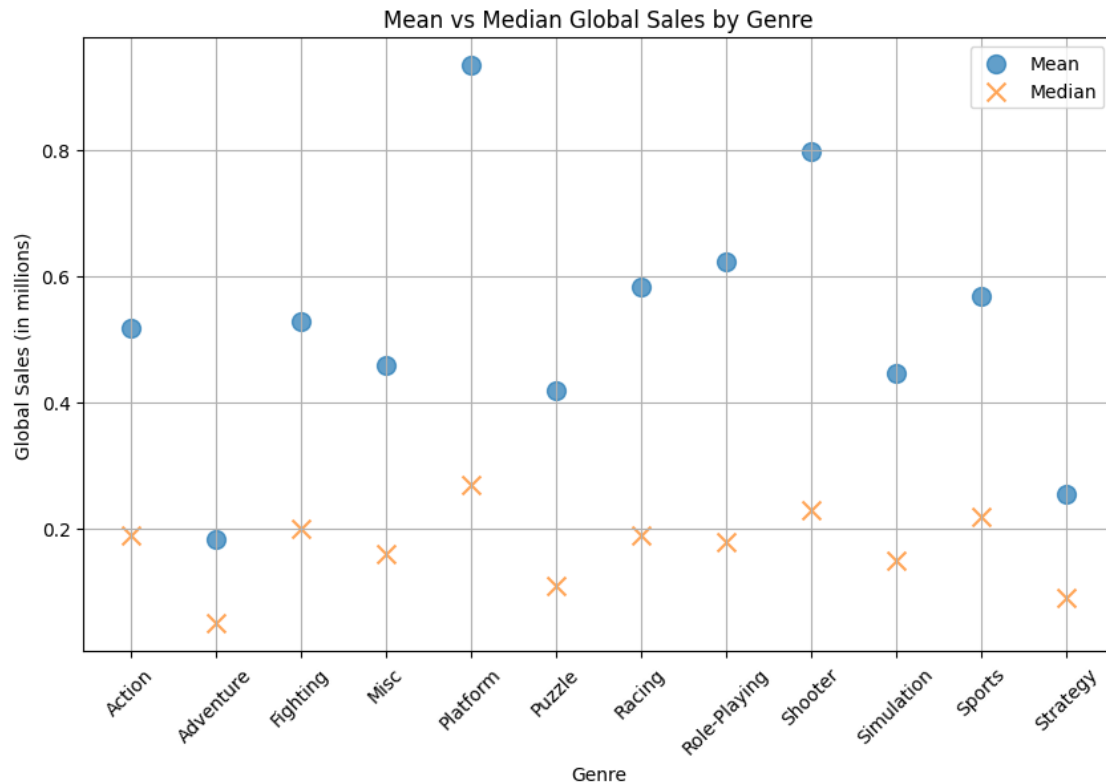
```



```
[27]: # Q2 Block 7: Line Plot of Average Global Sales by Genre
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genre_avg = df.groupby('Genre')['Global_Sales'].mean().
    ↪sort_values(ascending=False)
    plt.figure(figsize=(10, 6))
    plt.plot(genre_avg.index, genre_avg.values, marker='o')
    plt.title('Line Plot of Average Global Sales by Genre')
    plt.xlabel('Genre')
    plt.ylabel('Average Global Sales (in millions)')
    plt.xticks(rotation=45)
    plt.grid(True)
    plt.show()
else:
    print("Required columns 'Genre' or 'Global_Sales' not found.")
```



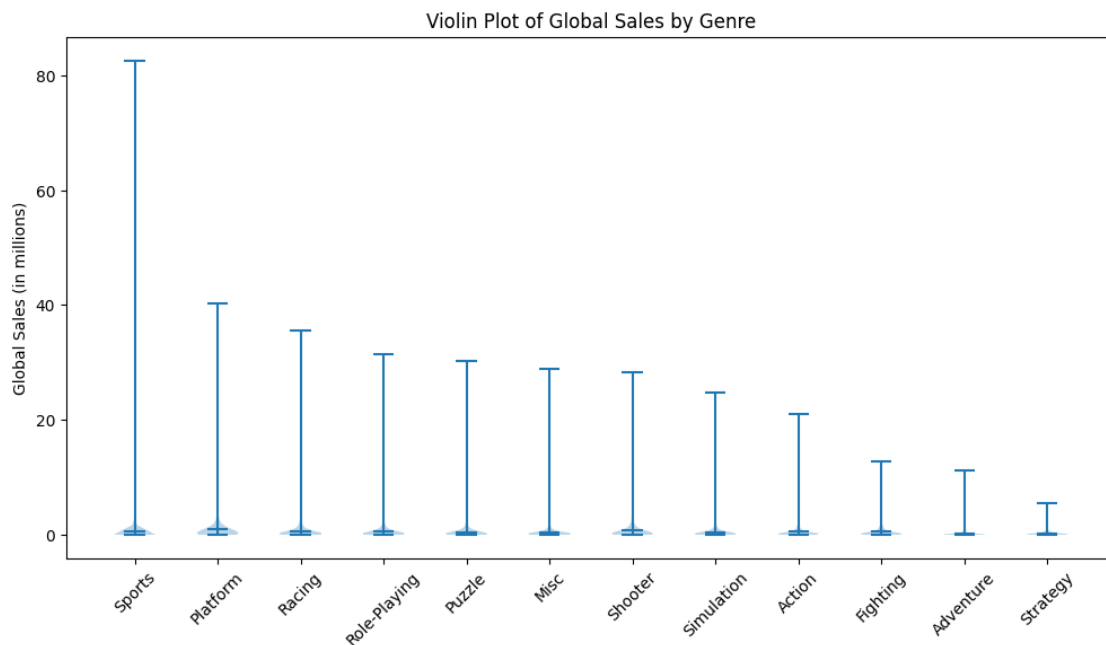
```
[28]: # Q2 Block 8: Scatter Plot Comparing Mean vs Median Global Sales by Genre
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    mean_sales = df.groupby('Genre')['Global_Sales'].mean()
    median_sales = df.groupby('Genre')['Global_Sales'].median()
    plt.figure(figsize=(10, 6))
    plt.scatter(mean_sales.index, mean_sales.values, label='Mean', s=100,
    ↪alpha=0.7)
    plt.scatter(median_sales.index, median_sales.values, label='Median', s=100,
    ↪alpha=0.7, marker='x')
    plt.title('Mean vs Median Global Sales by Genre')
    plt.xlabel('Genre')
    plt.ylabel('Global Sales (in millions)')
    plt.xticks(rotation=45)
    plt.legend()
    plt.grid(True)
    plt.show()
else:
    print("Required columns 'Genre' or 'Global_Sales' not found.")
```



```
[29]: # Q2 Block 9: Table of Genres Sorted by Average Global Sales with Count
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genre_stats = df.groupby('Genre').agg({'Global_Sales': ['mean', 'count']})
    genre_stats.columns = ['Average_Sales', 'Count']
    genre_stats = genre_stats.sort_values('Average_Sales', ascending=False)
    print(genre_stats.head(10))
else:
    print("Required columns 'Genre' or 'Global_Sales' not found.")
```

Genre	Average_Sales	Count
Platform	0.932523	888
Shooter	0.795873	1323
Role-Playing	0.622933	1500
Racing	0.583587	1249
Sports	0.567291	2348
Fighting	0.527067	849
Action	0.517884	3370
Misc	0.458960	1750
Simulation	0.446705	874
Puzzle	0.419000	580

```
[30]: # Q2 Block 10: Violin Plot of Global Sales by Genre
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genres = df['Genre'].dropna().unique()
    data = [df[df['Genre'] == genre]['Global_Sales'].dropna() for genre in
    genres]
    plt.figure(figsize=(12,6))
    plt.violinplot(data, showmeans=True)
    plt.xticks(range(1, len(genres)+1), genres, rotation=45)
    plt.title('Violin Plot of Global Sales by Genre')
    plt.ylabel('Global Sales (in millions)')
    plt.show()
else:
    print("Required columns not found.")
```



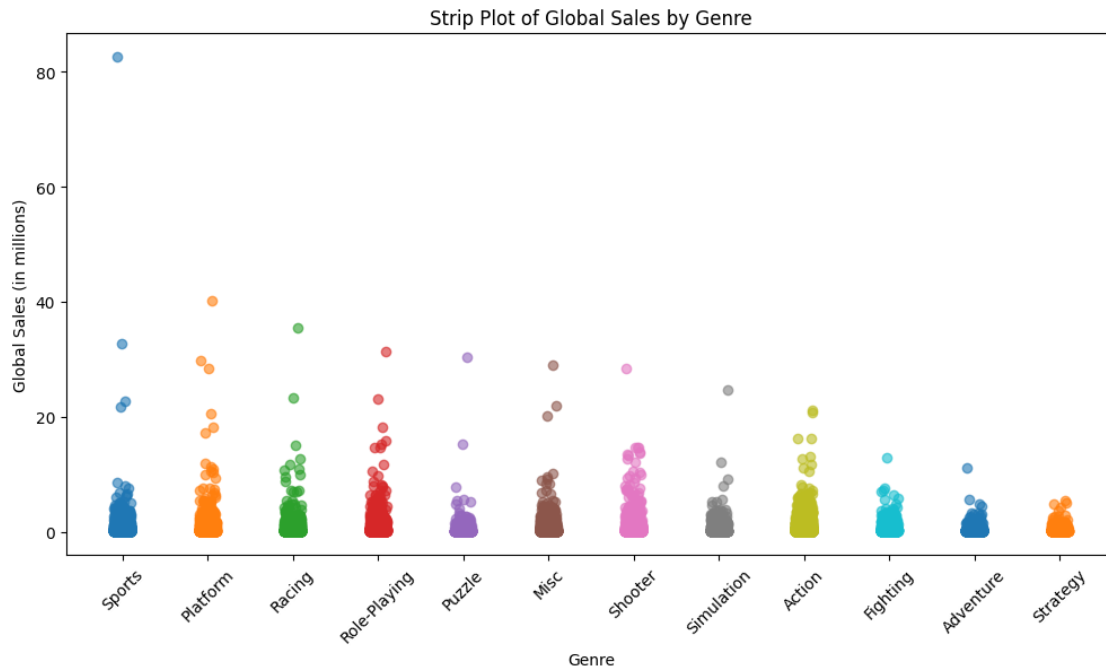
```
[31]: # Q2 Block 11: Strip Plot of Global Sales by Genre
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    plt.figure(figsize=(12,6))
    genres = df['Genre'].dropna().unique()
    for i, genre in enumerate(genres):
        sales = df[df['Genre'] == genre]['Global_Sales'].dropna()
        jitter = np.random.uniform(-0.1, 0.1, size=len(sales))
        plt.scatter(np.full_like(sales, i) + jitter, sales, alpha=0.6)
    plt.xticks(range(len(genres)), genres, rotation=45)
    plt.title('Strip Plot of Global Sales by Genre')
    plt.xlabel('Genre')
```



```

plt.ylabel('Global Sales (in millions)')
plt.show()
else:
    print("Required columns not found.")

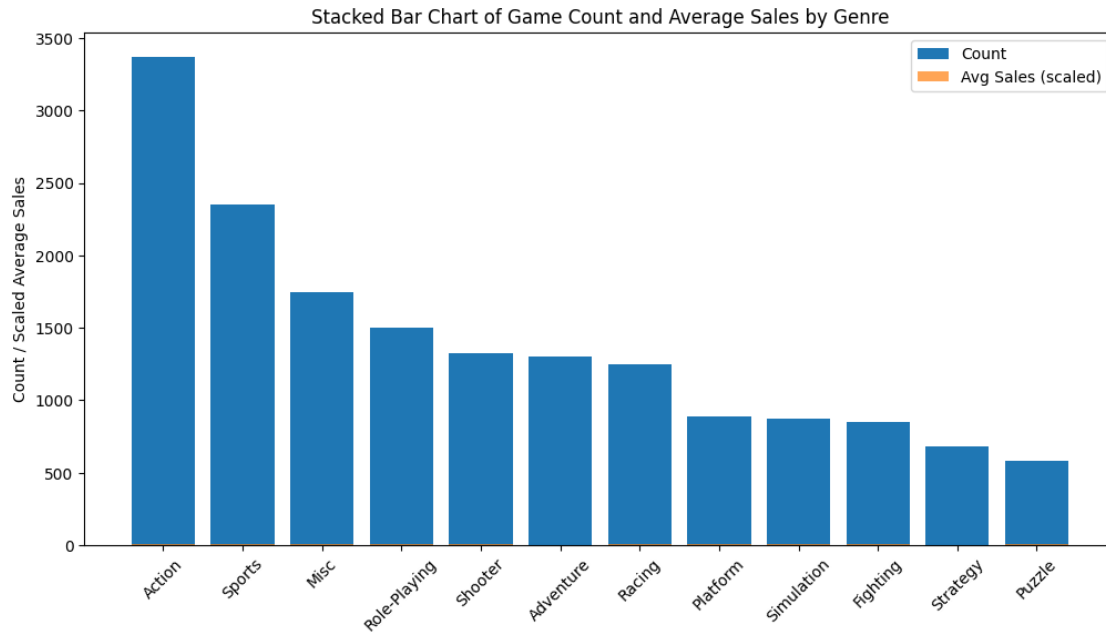
```



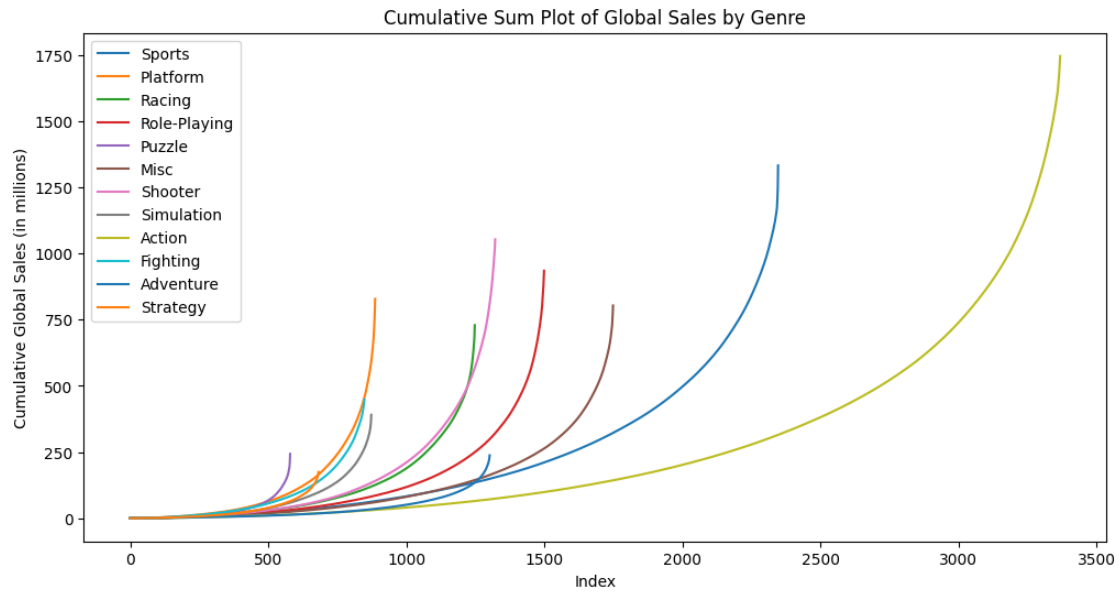
```

[32]: # Q2 Block 12: Stacked Bar Chart of Count of Games and Average Sales by Genre
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genre_counts = df['Genre'].value_counts()
    genre_avg = df.groupby('Genre')['Global_Sales'].mean()
    genres = genre_counts.index
    x = np.arange(len(genres))
    plt.figure(figsize=(12,6))
    plt.bar(x, genre_counts.loc[genres], label='Count')
    plt.bar(x, genre_avg.loc[genres]*10, alpha=0.7, label='Avg Sales (scaled)')
    plt.xticks(x, genres, rotation=45)
    plt.title('Stacked Bar Chart of Game Count and Average Sales by Genre')
    plt.ylabel('Count / Scaled Average Sales')
    plt.legend()
    plt.show()
else:
    print("Required columns not found.")

```

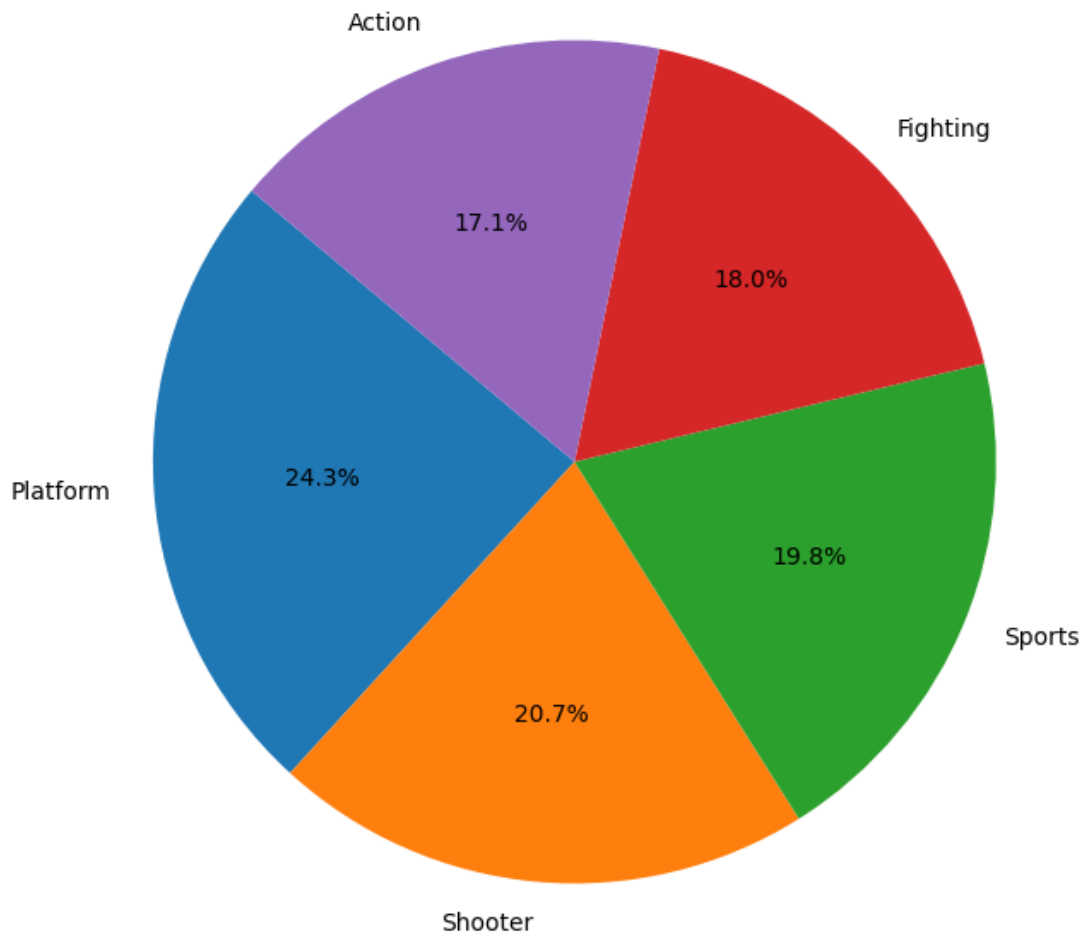


```
[33]: # Q2 Block 13: Cumulative Sum Plot of Global Sales by Genre
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    plt.figure(figsize=(12,6))
    genres = df['Genre'].dropna().unique()
    for genre in genres:
        sales_sorted = np.sort(df[df['Genre'] == genre]['Global_Sales'].
        ↪dropna())
        cumulative = np.cumsum(sales_sorted)
        plt.plot(cumulative, label=genre)
    plt.title('Cumulative Sum Plot of Global Sales by Genre')
    plt.xlabel('Index')
    plt.ylabel('Cumulative Global Sales (in millions)')
    plt.legend()
    plt.show()
else:
    print("Required columns not found.")
```



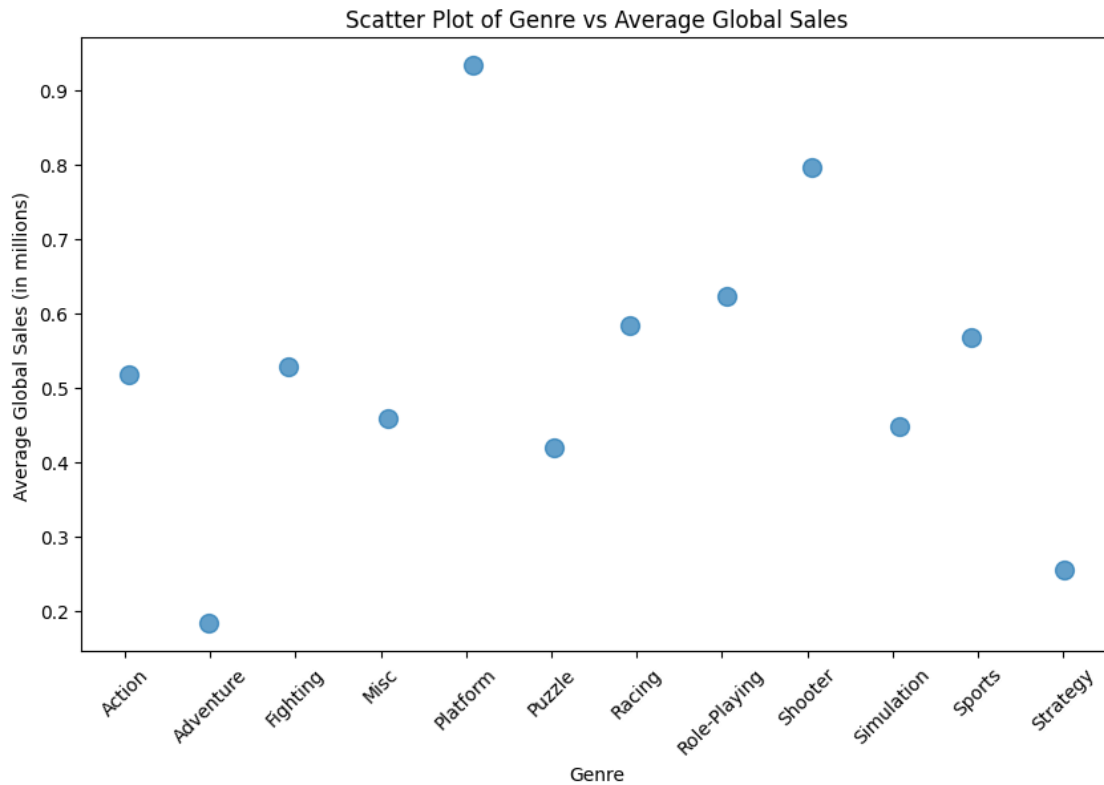
```
[34]: # Q2 Block 14: Pie Chart of Median Global Sales by Genre (Top 5)
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    median_sales = df.groupby('Genre')['Global_Sales'].median().
    ↪sort_values(ascending=False).head(5)
    plt.figure(figsize=(8,8))
    median_sales.plot(kind='pie', autopct='%1.1f%%', startangle=140)
    plt.title('Pie Chart of Top 5 Genres by Median Global Sales')
    plt.ylabel('')
    plt.show()
else:
    print("Required columns not found.")
```

Pie Chart of Top 5 Genres by Median Global Sales

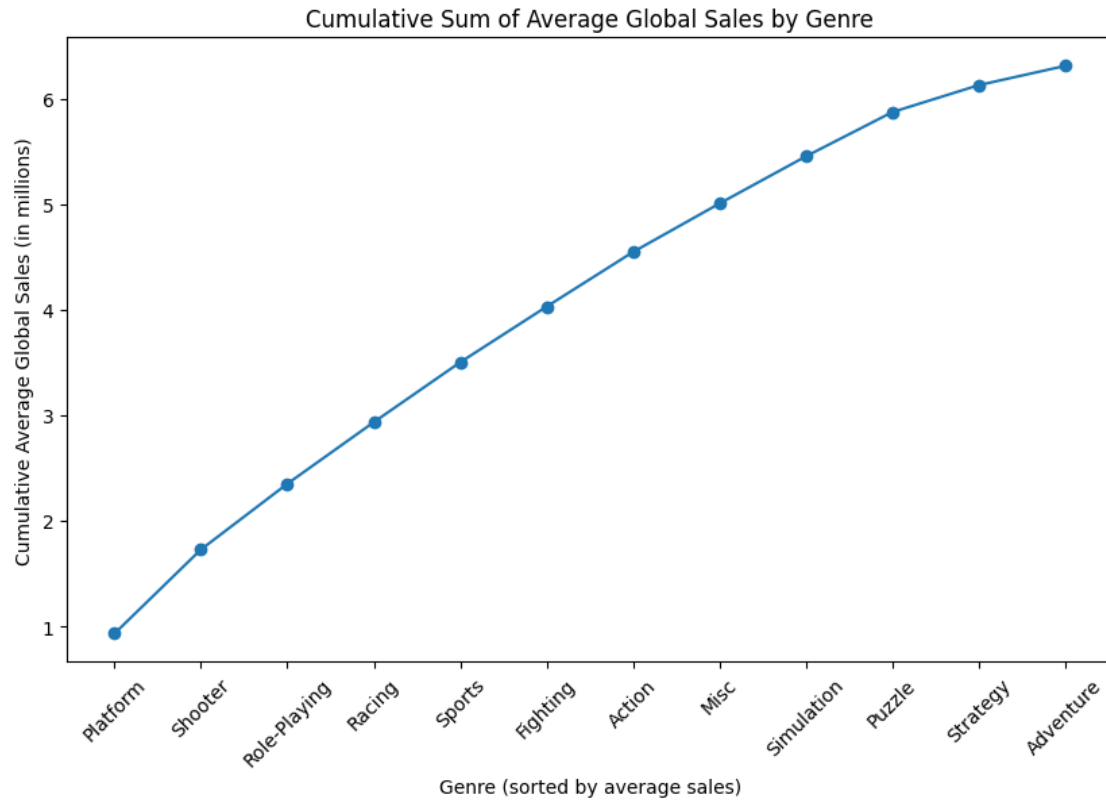


```
[35]: # Q2 Block 15: Scatter Plot of Genre vs Average Global Sales with Jitter
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genre_avg = df.groupby('Genre')['Global_Sales'].mean()
    genres = genre_avg.index
    jitter = np.random.uniform(-0.1, 0.1, size=len(genres))
    plt.figure(figsize=(10,6))
    plt.scatter(np.arange(len(genres)) + jitter, genre_avg.values, s=100,
alpha=0.7)
    plt.xticks(np.arange(len(genres)), genres, rotation=45)
    plt.title('Scatter Plot of Genre vs Average Global Sales')
    plt.xlabel('Genre')
    plt.ylabel('Average Global Sales (in millions)')
```

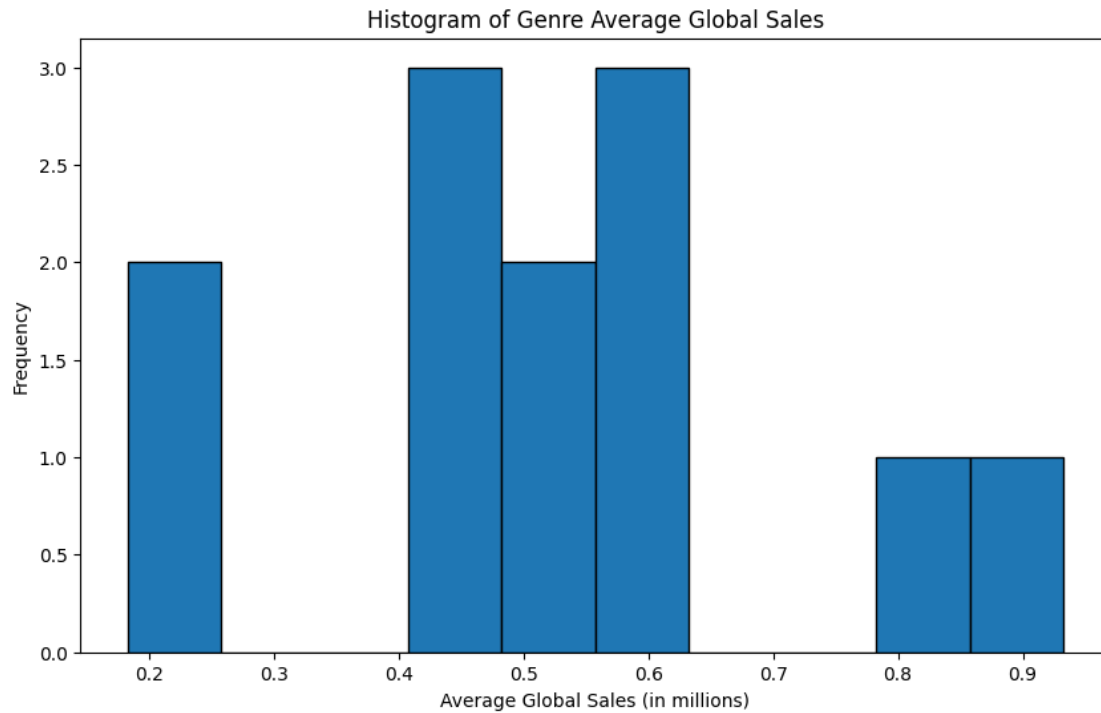
```
plt.show()
else:
    print("Required columns not found.")
```



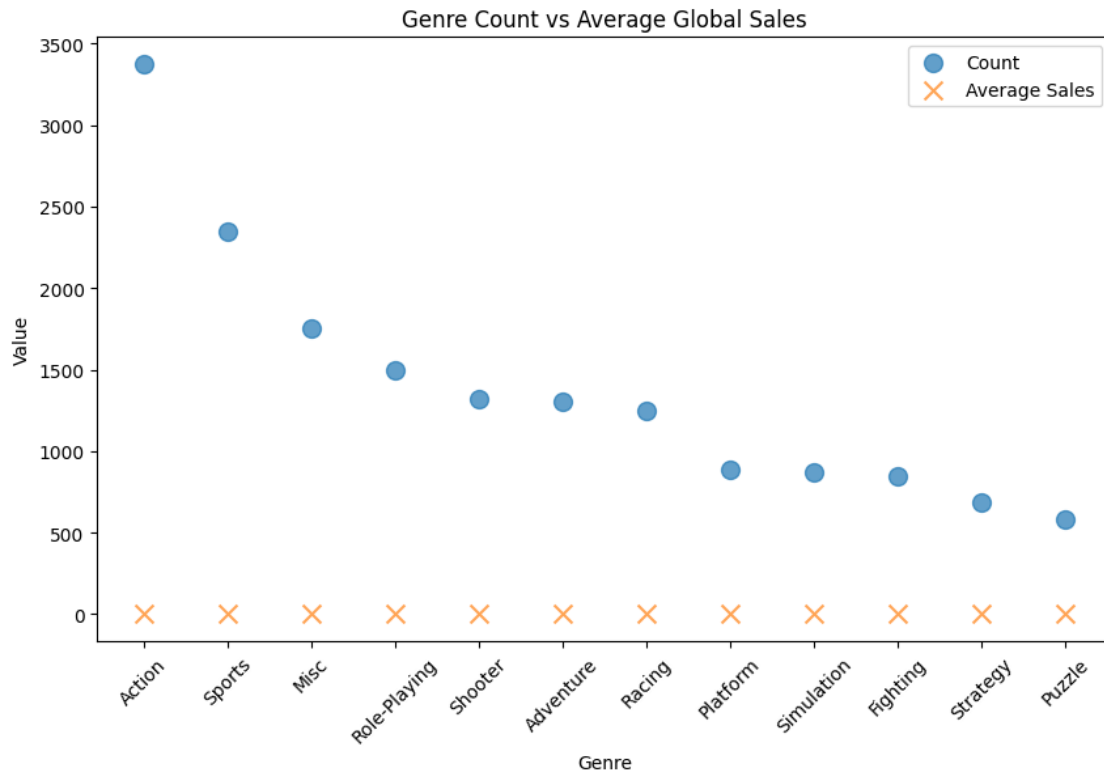
```
[36]: # Q2 Block 16: Line Plot of Cumulative Average Global Sales by Genre
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genre_avg = df.groupby('Genre')['Global_Sales'].mean().
    ↪sort_values(ascending=False)
    cumulative_avg = np.cumsum(genre_avg)
    plt.figure(figsize=(10,6))
    plt.plot(cumulative_avg, marker='o')
    plt.title('Cumulative Sum of Average Global Sales by Genre')
    plt.xlabel('Genre (sorted by average sales)')
    plt.ylabel('Cumulative Average Global Sales (in millions)')
    plt.xticks(ticks=np.arange(len(genre_avg)), labels=genre_avg.index,
    ↪rotation=45)
    plt.show()
else:
    print("Required columns not found.")
```



```
[37]: # Q2 Block 17: Histogram of Genre Average Global Sales
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genre_avg = df.groupby('Genre')['Global_Sales'].mean()
    plt.figure(figsize=(10,6))
    plt.hist(genre_avg, bins=10, edgecolor='k')
    plt.title('Histogram of Genre Average Global Sales')
    plt.xlabel('Average Global Sales (in millions)')
    plt.ylabel('Frequency')
    plt.show()
else:
    print("Required columns not found.")
```

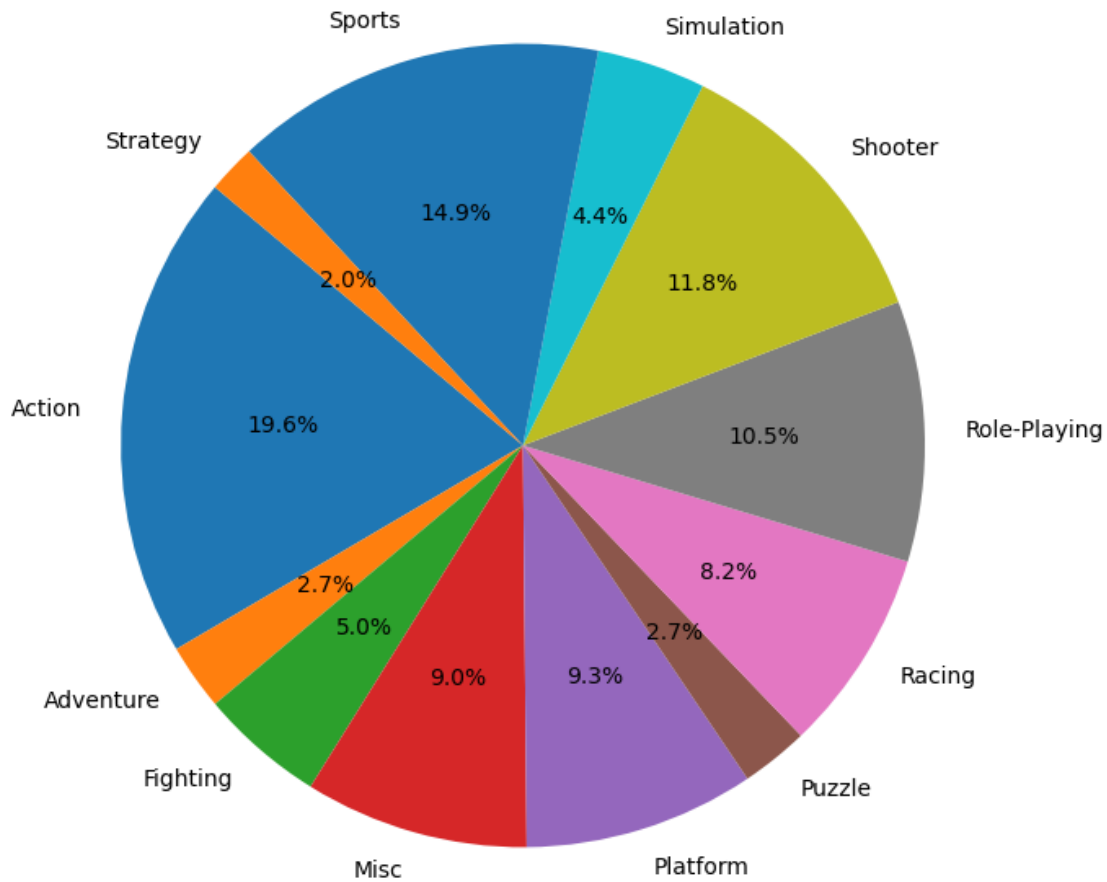


```
[38]: # Q2 Block 18: Scatter Plot of Genre Count vs Average Global Sales
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genre_count = df['Genre'].value_counts()
    genre_avg = df.groupby('Genre')['Global_Sales'].mean()
    plt.figure(figsize=(10,6))
    plt.scatter(genre_count.index, genre_count.values, s=100, label='Count',
    ↪alpha=0.7)
    plt.scatter(genre_avg.index, genre_avg.values, s=100, label='Average_
    ↪Sales', alpha=0.7, marker='x')
    plt.title('Genre Count vs Average Global Sales')
    plt.xlabel('Genre')
    plt.ylabel('Value')
    plt.legend()
    plt.xticks(rotation=45)
    plt.show()
else:
    print("Required columns not found.")
```



```
[39]: # Q2 Block 19: Pie Chart of Genre Share of Total Global Sales
if 'Genre' in df.columns and 'Global_Sales' in df.columns:
    genre_total = df.groupby('Genre')['Global_Sales'].sum()
    plt.figure(figsize=(8,8))
    genre_total.plot(kind='pie', autopct='%1.1f%%', startangle=140)
    plt.title('Pie Chart of Genre Share of Total Global Sales')
    plt.ylabel('')
    plt.show()
else:
    print("Required columns not found.")
```


Pie Chart of Genre Share of Total Global Sales



1.3 Question 3: How do Sales vary across different Platforms?

In this section, we analyze how Global Sales vary by Platform using multiple visualizations.

```
[40]: # Q3 Block 1: Group the data by Platform and Sum the Global Sales
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    platform_sales = df.groupby('Platform')['Global_Sales'].sum()
    print(platform_sales.sort_values(ascending=False))
else:
    print("Required columns 'Platform' or 'Global_Sales' not found.")
```

```
Platform
PS2      1255.64
X360     971.63
PS3      939.43
```

Wii	908.13
DS	807.10
PS	730.68
GBA	318.50
PS4	314.23
PSP	294.30
PC	260.30
3DS	259.09
XB	258.26
GB	255.45
NES	251.07
N64	218.88
SNES	200.05
GC	199.36
XOne	159.44
2600	97.08
WiiU	82.16
PSV	54.12
SAT	33.59
GEN	30.78
DC	15.97
SCD	1.87
NG	1.44
WS	1.42
TG16	0.16
3DO	0.10
GG	0.04
PCFX	0.03

Name: Global_Sales, dtype: float64

```
[41]: # Q3 Block 2: Display the Top 5 Platforms with Highest Total Global Sales
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    top_platforms = platform_sales.sort_values(ascending=False).head(5)
    print(top_platforms)
else:
    print("Required columns 'Platform' or 'Global_Sales' not found.")
```

Platform	
PS2	1255.64
X360	971.63
PS3	939.43
Wii	908.13
DS	807.10

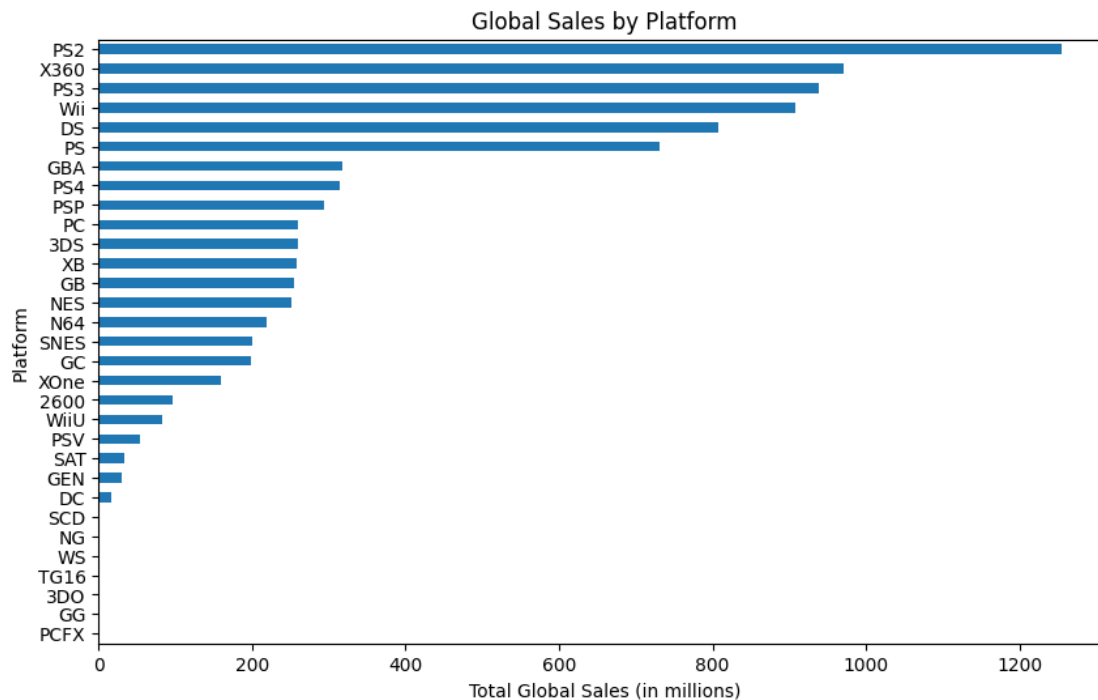
Name: Global_Sales, dtype: float64

```
[42]: # Q3 Block 3: Horizontal Bar Chart of Global Sales by Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    plt.figure(figsize=(10, 6))
```

```

platform_sales.sort_values().plot(kind='barh')
plt.title('Global Sales by Platform')
plt.xlabel('Total Global Sales (in millions)')
plt.ylabel('Platform')
plt.show()
else:
    print("Required columns 'Platform' or 'Global_Sales' not found.")

```

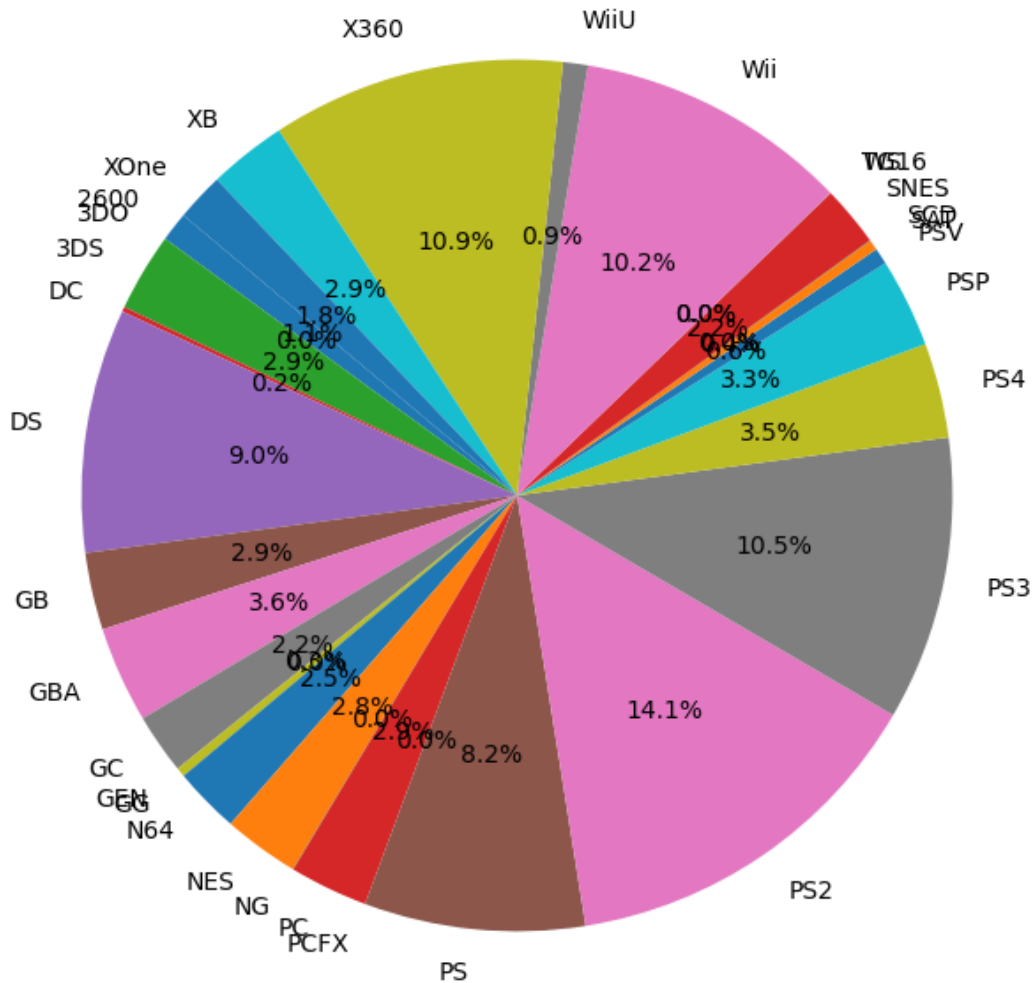


```

[43]: # Q3 Block 4: Pie Chart of Global Sales by Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    plt.figure(figsize=(8, 8))
    platform_sales.plot(kind='pie', autopct='%1.1f%%', startangle=140)
    plt.title('Pie Chart of Global Sales by Platform')
    plt.ylabel('')
    plt.show()
else:
    print("Required columns 'Platform' or 'Global_Sales' not found.")

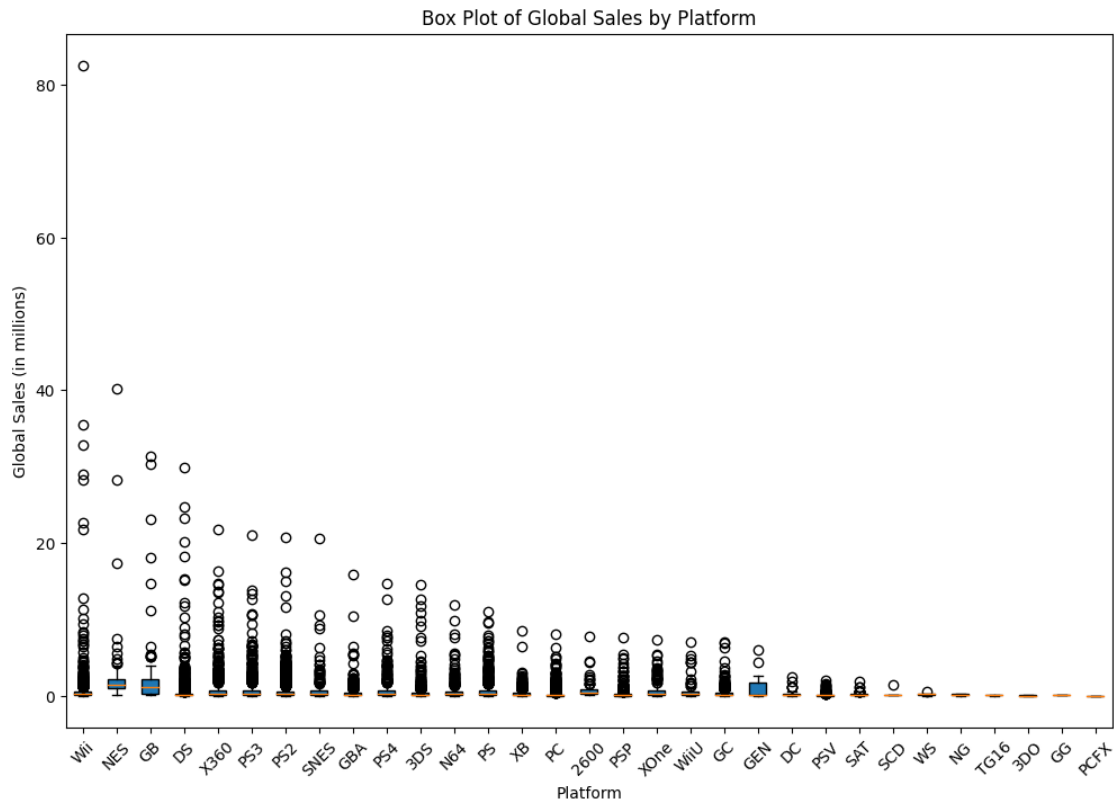
```

Pie Chart of Global Sales by Platform

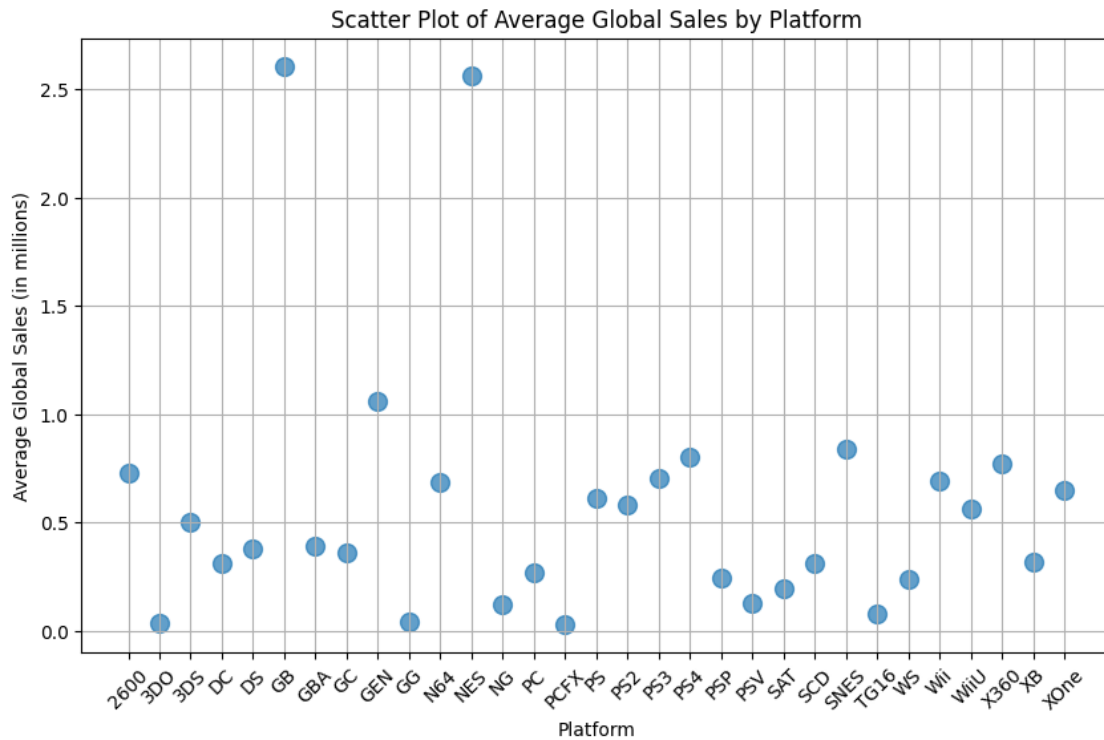


```
[44]: # Q3 Block 5: Box Plot of Global Sales for Each Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    platforms = df['Platform'].unique()
    data_to_plot = [df[df['Platform'] == platform]['Global_Sales'].dropna() for platform in platforms]
    plt.figure(figsize=(12, 8))
    plt.boxplot(data_to_plot, labels=platforms, patch_artist=True)
    plt.title('Box Plot of Global Sales by Platform')
    plt.xlabel('Platform')
    plt.ylabel('Global Sales (in millions)')
    plt.xticks(rotation=45)
```

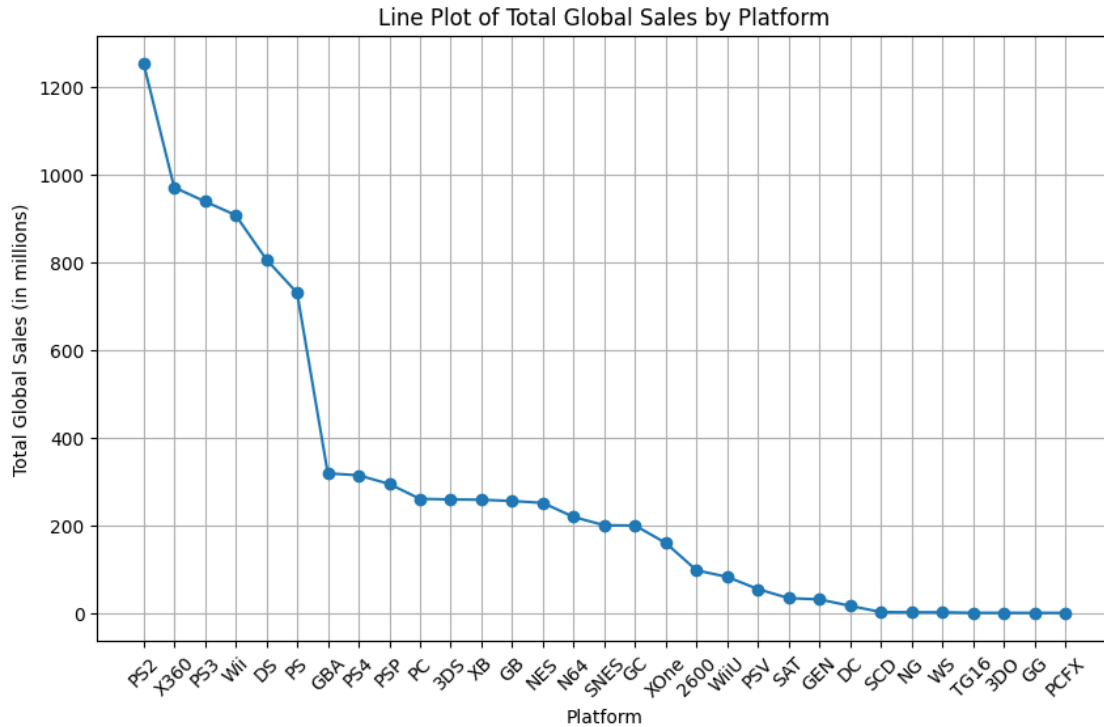
```
plt.show()
else:
    print("Required columns 'Platform' or 'Global_Sales' not found.")
```



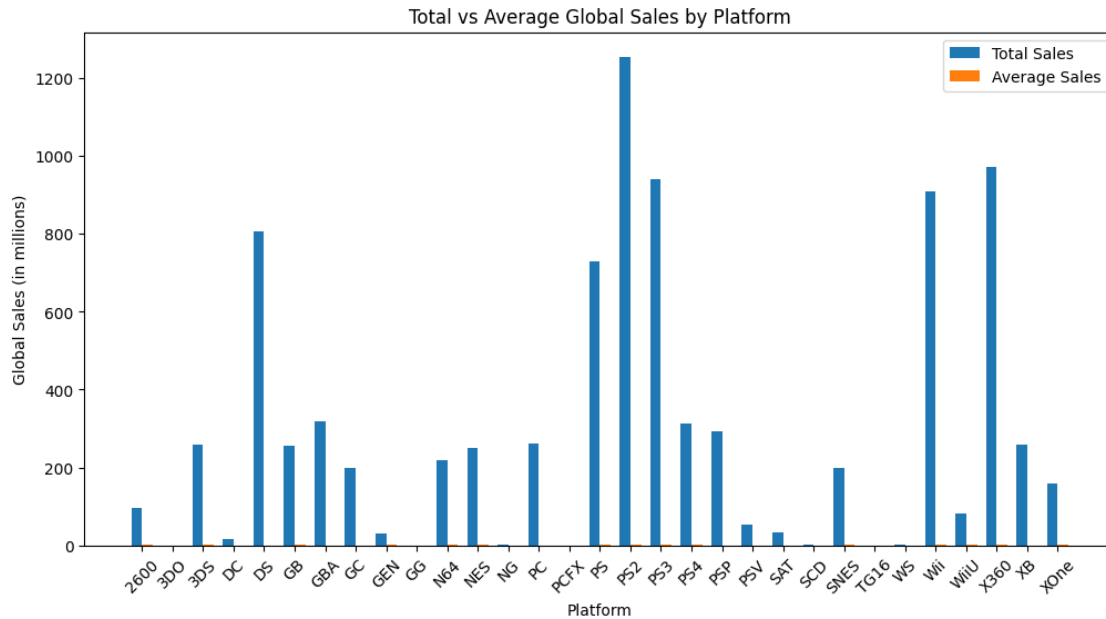
```
[45]: # Q3 Block 6: Scatter Plot of Average Global Sales by Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    platform_avg_sales = df.groupby('Platform')['Global_Sales'].mean()
    plt.figure(figsize=(10, 6))
    plt.scatter(platform_avg_sales.index, platform_avg_sales.values, s=100,
        alpha=0.7)
    plt.title('Scatter Plot of Average Global Sales by Platform')
    plt.xlabel('Platform')
    plt.ylabel('Average Global Sales (in millions)')
    plt.xticks(rotation=45)
    plt.grid(True)
    plt.show()
else:
    print("Required columns 'Platform' or 'Global_Sales' not found.")
```



```
[46]: # Q3 Block 7: Line Plot of Total Global Sales by Platform (Sorted)
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    sorted_platform_sales = platform_sales.sort_values(ascending=False)
    plt.figure(figsize=(10, 6))
    plt.plot(sorted_platform_sales.index, sorted_platform_sales.values,
             marker='o')
    plt.title('Line Plot of Total Global Sales by Platform')
    plt.xlabel('Platform')
    plt.ylabel('Total Global Sales (in millions)')
    plt.xticks(rotation=45)
    plt.grid(True)
    plt.show()
else:
    print("Required columns 'Platform' or 'Global_Sales' not found.")
```

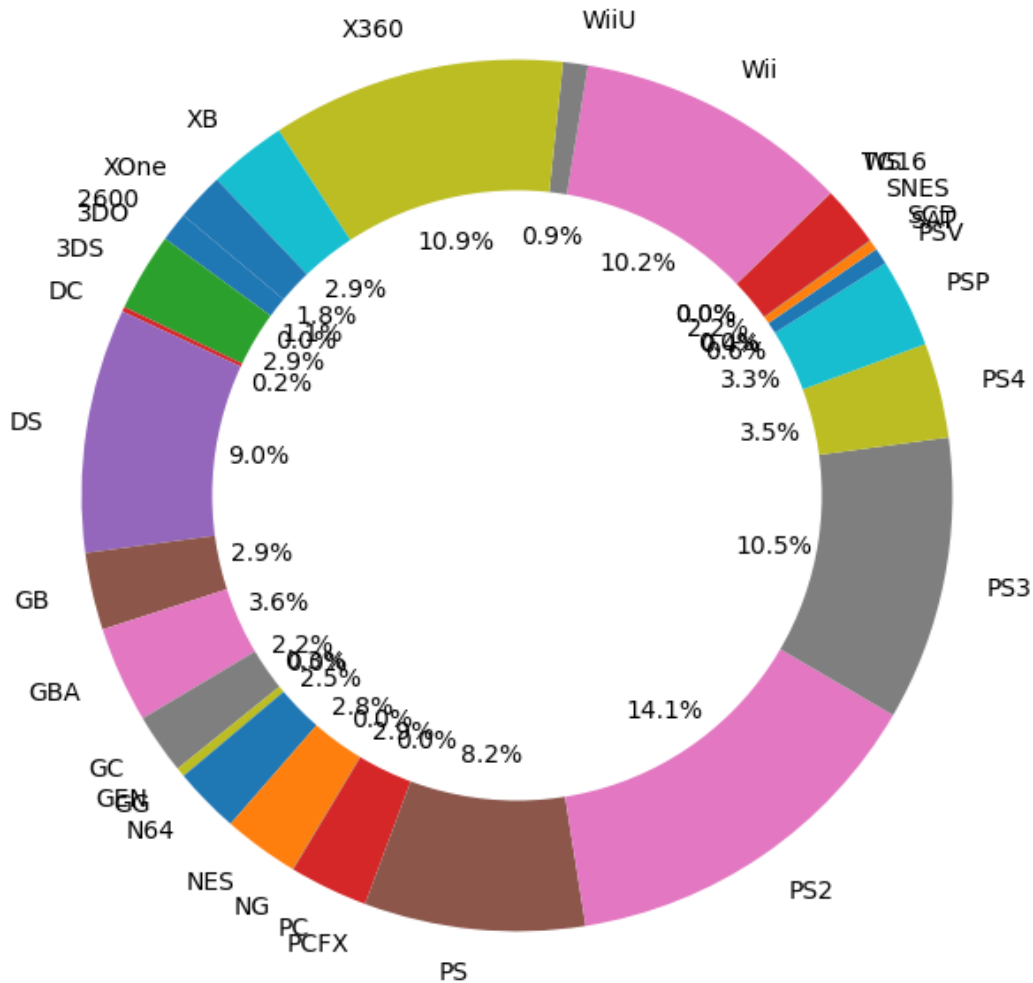


```
[47]: # Q3 Block 8: Grouped Bar Chart Comparing Total and Average Global Sales by Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    total_sales = df.groupby('Platform')['Global_Sales'].sum()
    avg_sales = df.groupby('Platform')['Global_Sales'].mean()
    platforms = total_sales.index
    x = np.arange(len(platforms))
    width = 0.35
    plt.figure(figsize=(12, 6))
    plt.bar(x - width/2, total_sales.loc[platforms], width, label='Total Sales')
    plt.bar(x + width/2, avg_sales.loc[platforms], width, label='Average Sales')
    plt.xlabel('Platform')
    plt.ylabel('Global Sales (in millions)')
    plt.title('Total vs Average Global Sales by Platform')
    plt.xticks(x, platforms, rotation=45)
    plt.legend()
    plt.show()
else:
    print("Required columns 'Platform' or 'Global_Sales' not found.")
```



```
[48]: # Q3 Block 9: Donut Chart of Global Sales Percentage by Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    total_sales = df.groupby('Platform')['Global_Sales'].sum()
    plt.figure(figsize=(8,8))
    wedges, texts, autotexts = plt.pie(total_sales, labels=total_sales.index,
    ↪ autopct='%1.1f%%', startangle=140)
    centre_circle = plt.Circle((0,0),0.70,fc='white')
    fig = plt.gcf()
    fig.gca().add_artist(centre_circle)
    plt.title('Donut Chart of Global Sales Percentage by Platform')
    plt.show()
else:
    print("Required columns not found.")
```


Donut Chart of Global Sales Percentage by Platform

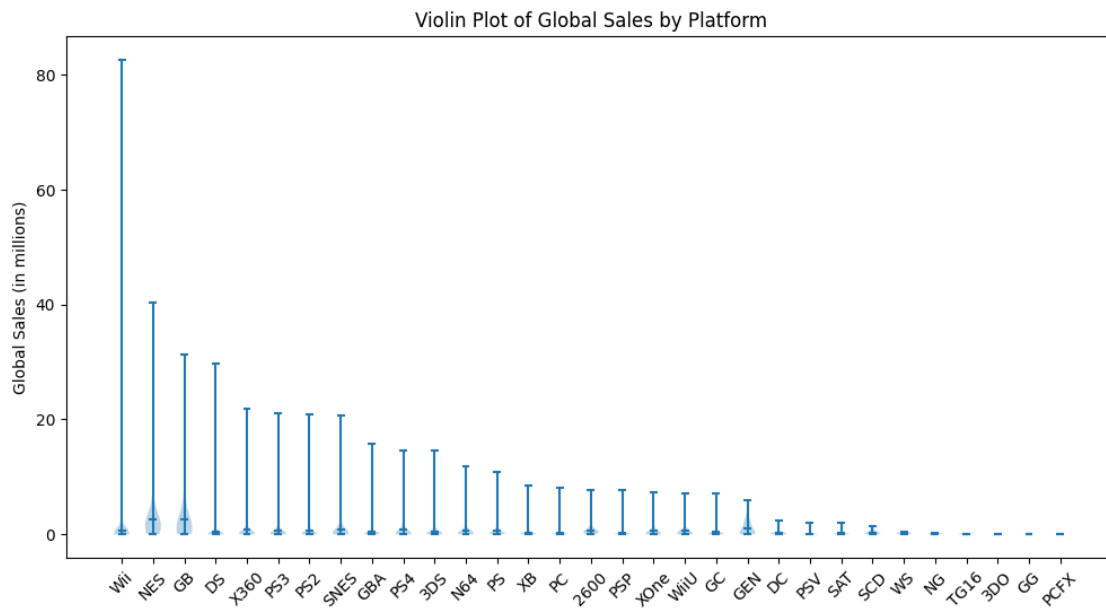


```
[49]: # Q3 Block 10: Violin Plot of Global Sales by Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    platforms = df['Platform'].dropna().unique()
    data = [df[df['Platform'] == platform]['Global_Sales'].dropna() for platform in platforms]
    plt.figure(figsize=(12,6))
    plt.violinplot(data, showmeans=True)
    plt.xticks(range(1, len(platforms)+1), platforms, rotation=45)
    plt.title('Violin Plot of Global Sales by Platform')
    plt.ylabel('Global Sales (in millions)')
    plt.show()
```

```

else:
    print("Required columns not found.")

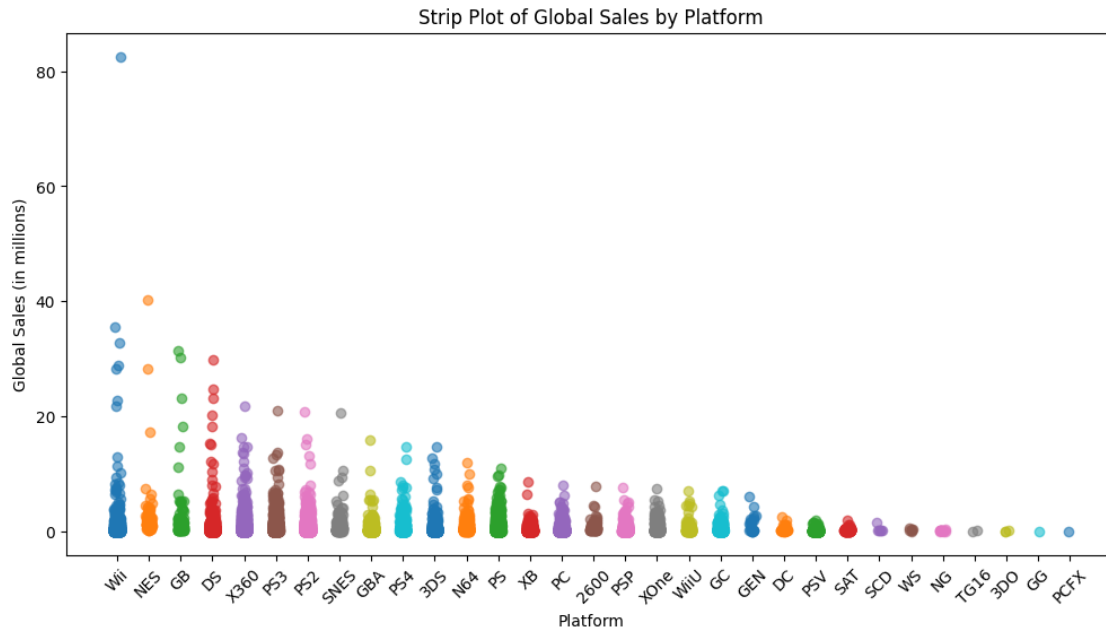
```



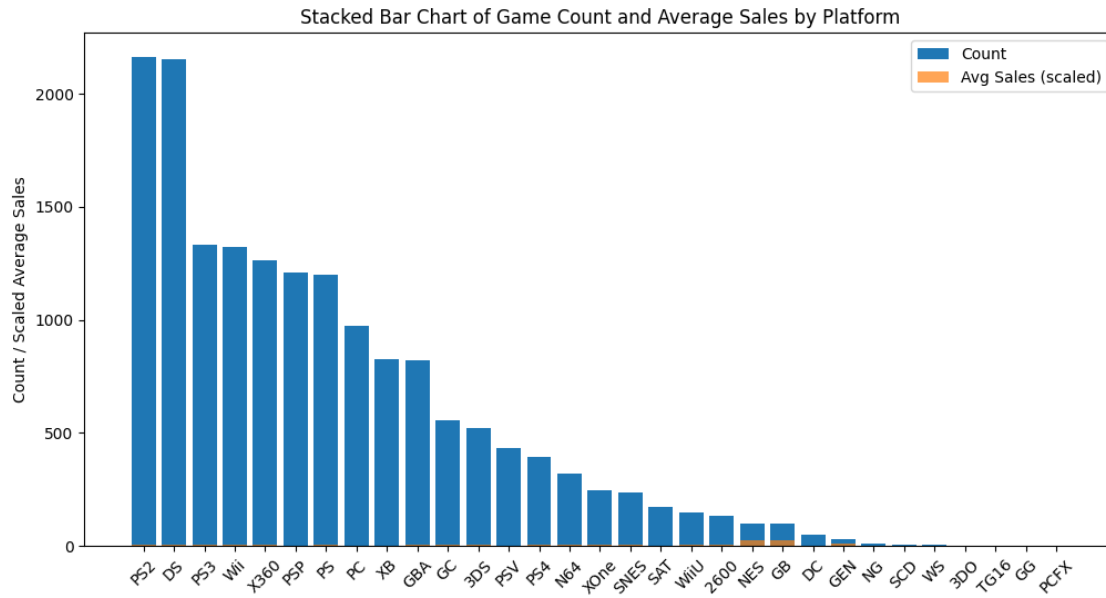
```

[50]: # Q3 Block 11: Strip Plot of Global Sales by Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    plt.figure(figsize=(12,6))
    platforms = df['Platform'].dropna().unique()
    for i, platform in enumerate(platforms):
        sales = df[df['Platform'] == platform]['Global_Sales'].dropna()
        jitter = np.random.uniform(-0.1, 0.1, size=len(sales))
        plt.scatter(np.full_like(sales, i) + jitter, sales, alpha=0.6)
    plt.xticks(range(len(platforms)), platforms, rotation=45)
    plt.title('Strip Plot of Global Sales by Platform')
    plt.xlabel('Platform')
    plt.ylabel('Global Sales (in millions)')
    plt.show()
else:
    print("Required columns not found.")

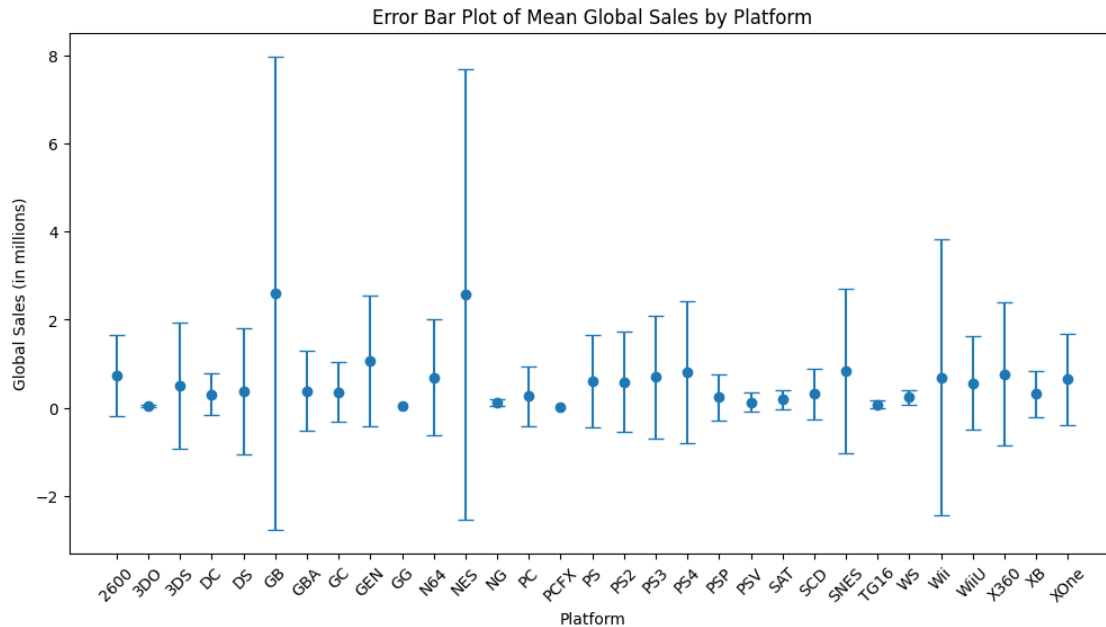
```



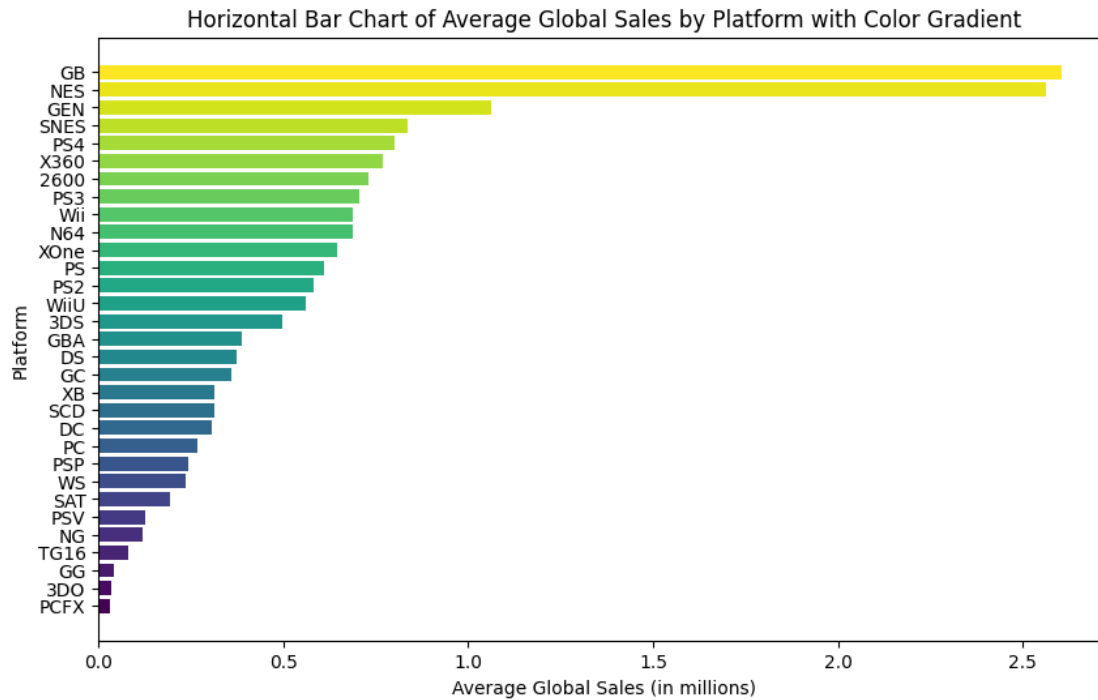
```
[51]: # Q3 Block 12: Stacked Bar Chart of Count of Games and Average Sales by Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    platform_counts = df['Platform'].value_counts()
    platform_avg = df.groupby('Platform')['Global_Sales'].mean()
    platforms = platform_counts.index
    x = np.arange(len(platforms))
    plt.figure(figsize=(12,6))
    plt.bar(x, platform_counts.loc[platforms], label='Count')
    plt.bar(x, platform_avg.loc[platforms]*10, alpha=0.7, label='Avg Sales_
↳(scaled)')
    plt.xticks(x, platforms, rotation=45)
    plt.title('Stacked Bar Chart of Game Count and Average Sales by Platform')
    plt.ylabel('Count / Scaled Average Sales')
    plt.legend()
    plt.show()
else:
    print("Required columns not found.")
```



```
[52]: # Q3 Block 13: Error Bar Plot of Mean Global Sales with Standard Deviation by Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    platform_group = df.groupby('Platform')['Global_Sales']
    means = platform_group.mean()
    stds = platform_group.std()
    platforms = means.index
    x = np.arange(len(platforms))
    plt.figure(figsize=(12,6))
    plt.errorbar(x, means, yerr=stds, fmt='o', capsize=5)
    plt.xticks(x, platforms, rotation=45)
    plt.title('Error Bar Plot of Mean Global Sales by Platform')
    plt.xlabel('Platform')
    plt.ylabel('Global Sales (in millions)')
    plt.show()
else:
    print("Required columns not found.")
```

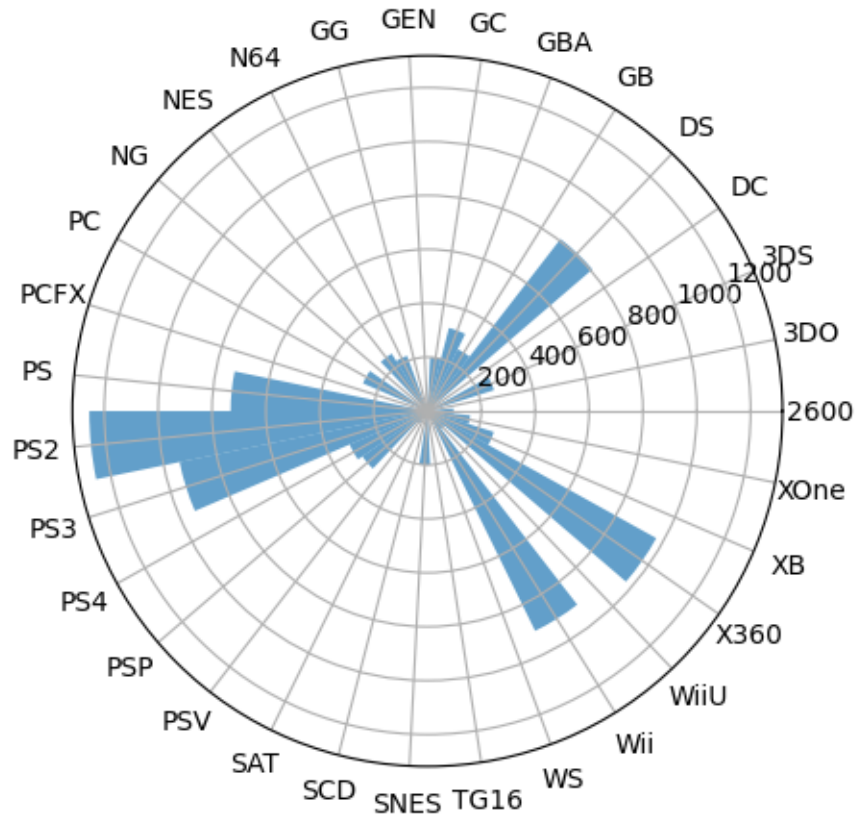


```
[53]: # Q3 Block 14: Horizontal Bar Chart of Average Global Sales by Platform with
      ↪ Color Gradient
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    avg_sales = df.groupby('Platform')['Global_Sales'].mean().sort_values()
    plt.figure(figsize=(10,6))
    bars = plt.barh(avg_sales.index, avg_sales.values, color=plt.cm.viridis(np.
    ↪ linspace(0,1,len(avg_sales))))
    plt.title('Horizontal Bar Chart of Average Global Sales by Platform with
    ↪ Color Gradient')
    plt.xlabel('Average Global Sales (in millions)')
    plt.ylabel('Platform')
    plt.show()
else:
    print("Required columns not found.")
```

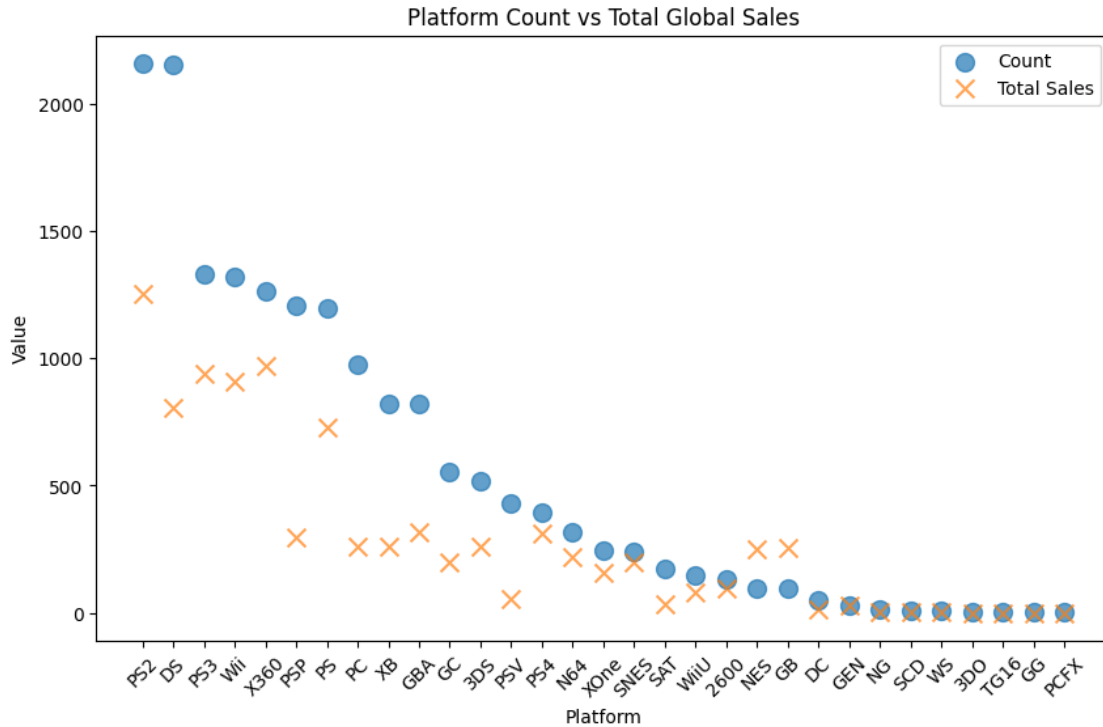


```
[54]: # Q3 Block 15: Polar Bar Chart of Total Global Sales by Platform
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    total_sales = df.groupby('Platform')['Global_Sales'].sum()
    platforms = total_sales.index
    sales = total_sales.values
    theta = np.linspace(0.0, 2 * np.pi, len(platforms), endpoint=False)
    width = 2 * np.pi / len(platforms)
    ax = plt.subplot(111, polar=True)
    bars = ax.bar(theta, sales, width=width, bottom=0.0, alpha=0.7)
    ax.set_xticks(theta)
    ax.set_xticklabels(platforms)
    plt.title('Polar Bar Chart of Total Global Sales by Platform', y=1.08)
    plt.show()
else:
    print("Required columns not found.")
```

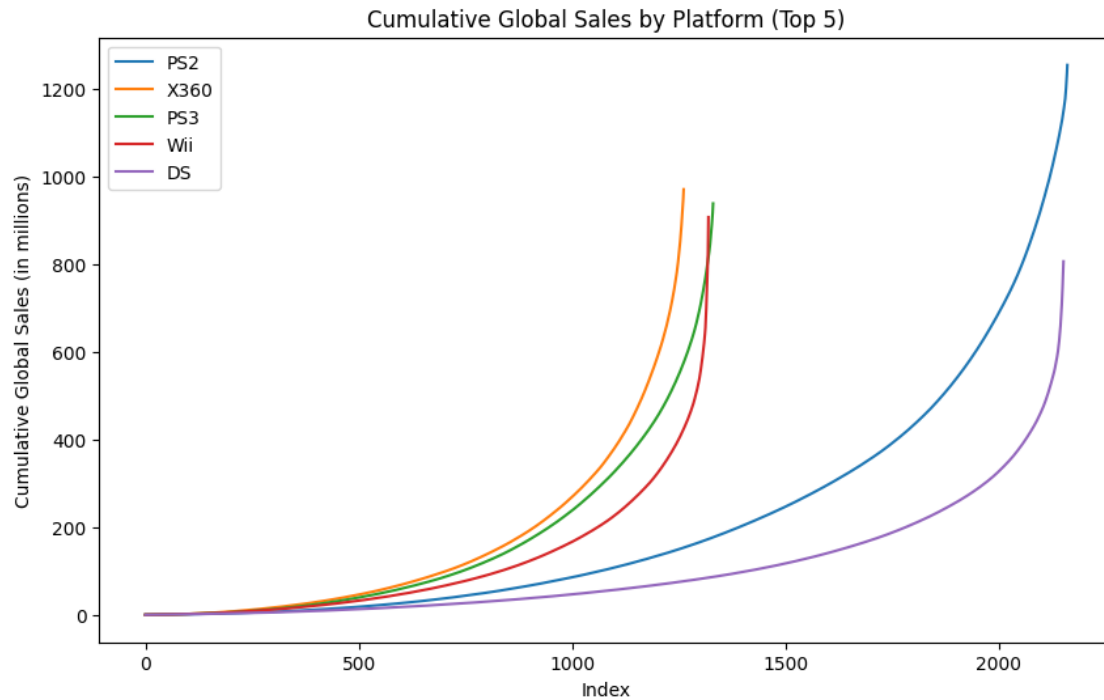
Polar Bar Chart of Total Global Sales by Platform



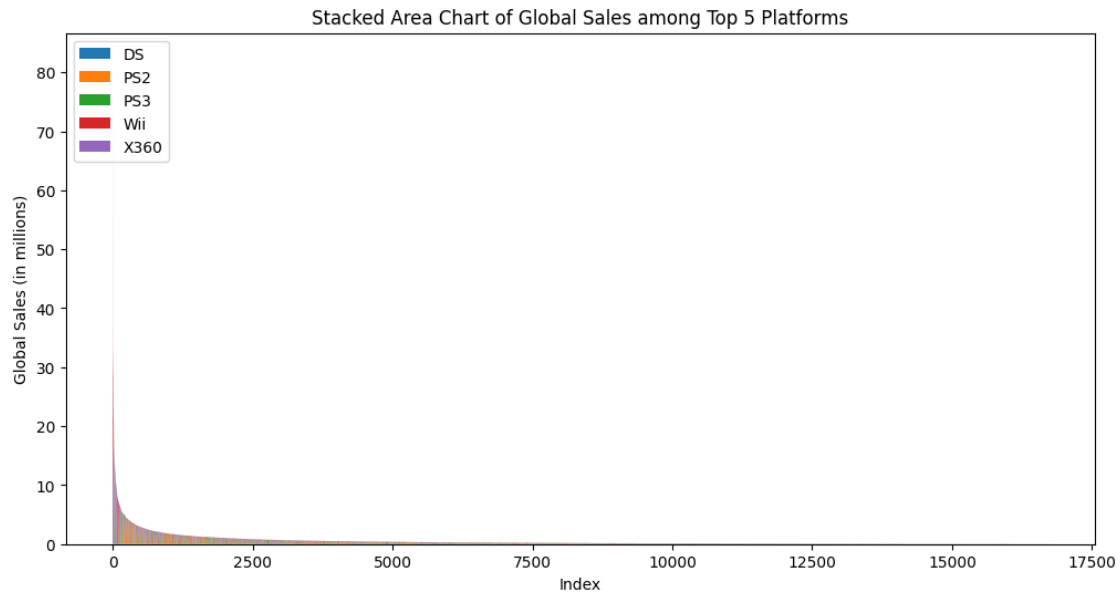
```
[55]: # Q3 Block 16: Scatter Plot of Platform Count vs Total Global Sales
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    platform_count = df['Platform'].value_counts()
    total_sales = df.groupby('Platform')['Global_Sales'].sum()
    plt.figure(figsize=(10,6))
    plt.scatter(platform_count.index, platform_count.values, s=100,
    label='Count', alpha=0.7)
    plt.scatter(total_sales.index, total_sales.values, s=100, label='Total_
    Sales', alpha=0.7, marker='x')
    plt.title('Platform Count vs Total Global Sales')
    plt.xlabel('Platform')
    plt.ylabel('Value')
    plt.legend()
    plt.xticks(rotation=45)
    plt.show()
else:
    print("Required columns not found.")
```



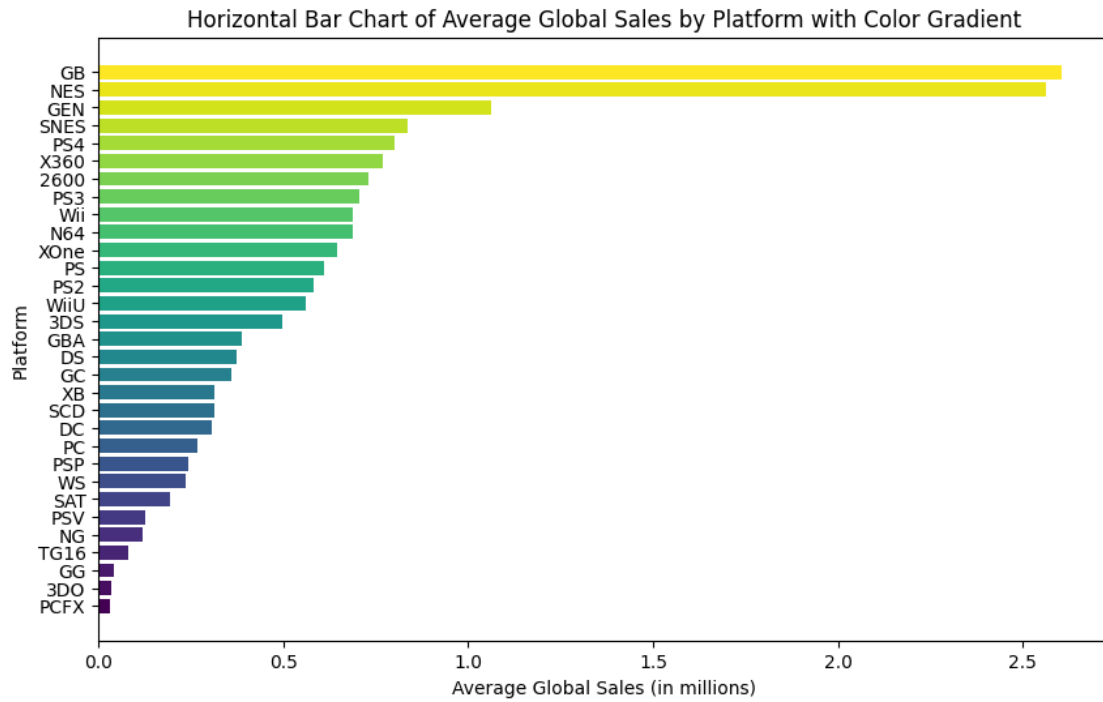
```
[56]: # Q3 Block 17: Line Plot of Cumulative Global Sales by Platform (Top 5)
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    total_sales = df.groupby('Platform')['Global_Sales'].sum().
    ↪sort_values(ascending=False).head(5)
    plt.figure(figsize=(10,6))
    for platform in total_sales.index:
        sales = df[df['Platform'] == platform]['Global_Sales'].dropna().values
        sales_sorted = np.sort(sales)
        cumulative = np.cumsum(sales_sorted)
        plt.plot(cumulative, label=platform)
    plt.title('Cumulative Global Sales by Platform (Top 5)')
    plt.xlabel('Index')
    plt.ylabel('Cumulative Global Sales (in millions)')
    plt.legend()
    plt.show()
else:
    print("Required columns not found.")
```

```
[57]: # Q3 Block 18: Stacked Area Chart of Global Sales Distribution among Top 5
      ↪ Platforms
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    top5 = df['Platform'].value_counts().head(5).index
    subset = df[df['Platform'].isin(top5)]
    subset_sorted = subset.sort_values('Platform')
    pivot = subset_sorted.pivot_table(index=subset_sorted.index,
    ↪ columns='Platform', values='Global_Sales', fill_value=0)
    plt.figure(figsize=(12,6))
    plt.stackplot(pivot.index, pivot.T, labels=pivot.columns)
    plt.title('Stacked Area Chart of Global Sales among Top 5 Platforms')
    plt.xlabel('Index')
    plt.ylabel('Global Sales (in millions)')
    plt.legend(loc='upper left')
    plt.show()
else:
    print("Required columns not found.")
```



```
[58]: # Q3 Block 19: Horizontal Bar Chart of Average Global Sales by Platform with
      ↪Color Gradient
if 'Platform' in df.columns and 'Global_Sales' in df.columns:
    avg_sales = df.groupby('Platform')['Global_Sales'].mean().sort_values()
    plt.figure(figsize=(10,6))
    bars = plt.barh(avg_sales.index, avg_sales.values, color=plt.cm.viridis(np.
    ↪linspace(0,1,len(avg_sales))))
    plt.title('Horizontal Bar Chart of Average Global Sales by Platform with
    ↪Color Gradient')
    plt.xlabel('Average Global Sales (in millions)')
    plt.ylabel('Platform')
    plt.show()
else:
    print("Required columns not found.")
```



1.4 Conclusion

This notebook provided a comprehensive analysis of the Video Games Sales dataset by exploring three questions:

- The distribution of Global Sales (19 diverse blocks)
- The average Global Sales by Genre (19 diverse blocks)
- The variation of Global Sales across Platforms (19 diverse blocks)

Feel free to extend or modify the analysis further!